

SOFTWARE ENGINEERING INTERVIEW PREPARATION

CodeTrackr

Complete technical analysis, architecture reference, and
interview question bank — reconstructed from the actual
codebase

Repository: `CodeTrackr-main` · branch `feat/security-and-insights`

Analysis date: 2026-09-07

Companion documents: [Architecture](#) · [Interview Q&A](#) · [Cheat Sheet](#) · [Project Context](#)

Contents

1. Project overview & elevator pitches

What it is [ACTUAL IMPLEMENTATION]

The one honest caveat to lead with

30-second version

1-minute version

2-minute version

5-minute deep technical version

"Tell me about your project."

"What was your contribution?" [ground this ONLY in repo evidence]

Why I built it

What problem it solves / what makes it different

Hardest part / biggest technical challenge

What I'd improve

2. Architecture

Shape [ACTUAL IMPLEMENTATION]

Why this architecture

Request lifecycles

3. Technology stack (with alternatives & trade-offs)

React 19 + Vite 7 + TypeScript

Node.js 18 + Express 5

MongoDB + Mongoose 8

VS Code Extension API (`@types/vscode ^1.85`) + esbuild

JSON Web Tokens (`jsonwebtoken`) + Passport (`passport-google-oauth20`) + `cookie-parser`

`crypto` (Node built-in) — API keys & group passwords

MongoDB driver via Mongoose — no ORM/query builder beyond it. `node-cron` for scheduling.

Chart.js 4 + react-chartjs-2 + chartjs-plugin-datalabels

Tailwind CSS 3 + a bespoke `ThemeContext`

Deployment: Vercel (frontend) + Render (backend) + MongoDB Atlas

Not used (and you'll be asked why): Redis, Kafka/SQS, WebSockets, GraphQL, Docker, any CI.

4. The VS Code extension — in depth

4.1 Installation → activation

4.2 The flush loop and idle/pause state machine

4.3 What is collected, and how

4.4 Frequency, batching, buffering, debounce/throttle

4.5 Lifecycle edge cases [ACTUAL IMPLEMENTATION]

4.6 Where the API key lives locally

4.7 Source-file map (extension)

5. The unique-key / account-linking system

5.1 End to end [ACTUAL IMPLEMENTATION]

5.2 Is this actually "just an identifier"? — the discussion an interviewer wants

5.3 Attack scenarios [ACTUAL IMPLEMENTATION risk]

5.4 How to redesign it [RECOMMENDED IMPROVEMENT]

6. Database design (MongoDB / Mongoose)

6.1 Collections, purpose, structure

6.2 Indexes [ACTUAL IMPLEMENTATION + gap]

6.3 SQL vs MongoDB — the full answer

6.4 Consistency, races, atomicity

7. Backend & API architecture

7.1 Endpoint table

7.2 Middleware pipeline

7.3 Status codes actually used

7.4 API design weaknesses

8. Frontend architecture

8.1 Structure

8.2 State management [ACTUAL IMPLEMENTATION]

8.3 React concepts actually used (and the questions they invite)

8.4 Data flow (frontend)

8.5 Loading / error / empty states

8.6 Known frontend problems [ACTUAL IMPLEMENTATION]

9. Analytics processing

9.1 Where it happens [ACTUAL IMPLEMENTATION]

9.2 The daily endpoint, step by step

9.3 Timezone handling

9.4 Correctness history (all in `IMPROVEMENT_PLAN.md` , all fixed)

9.5 Complexity

10. Leaderboard

10.1 What determines rank [ACTUAL IMPLEMENTATION]

10.2 How it's computed

10.3 Properties & problems

10.4 Complexity

10.5 "Optimise for 10M users" [RECOMMENDED IMPROVEMENT]

11. Groups

11.1 Model [ACTUAL IMPLEMENTATION]

11.2 Lifecycle

11.3 Authorization & concurrency issues

11.4 Interview questions this raises

12. Insights / ML system

12.1 The honest classification [ACTUAL IMPLEMENTATION]

12.2 The pipeline

12.3 Every metric, defined exactly

12.4 Sync/async, caching, regeneration

12.5 "Is this really machine learning?" — the answer

12.6 If asked to "make it ML" [RECOMMENDED IMPROVEMENT]

13. Authentication & authorization

13.1 Web auth flow [ACTUAL IMPLEMENTATION]

13.2 Extension auth

13.3 Authorization [ACTUAL IMPLEMENTATION]

13.4 Gaps

14. Security analysis

14.1 API key — plaintext bearer credential

14.2 No rate limiting anywhere

14.3 No security headers

14.4 No input validation on ingest

14.5 Error messages leak internals

14.6 Leaderboard exposes every user's email

14.7 AUTH_BYPASS — fixed, but note it

14.8 JWT

14.9 No idempotency / replay protection on ingest

14.10 MongoDB injection

14.11 XSS / CSRF

14.12 Extension security

15. Error handling & reliability

16. Performance & scalability (10k → 1M users)

10,000 users [ACTUAL: mostly fine]

100,000 users

1,000,000 users

What to say

17. Testing

17.1 What exists [ACTUAL IMPLEMENTATION]

17.2 Gaps

17.3 Recommended strategy [RECOMMENDED IMPROVEMENT]

18. Debugging playbook

18.1 "A user's VS Code activity isn't on the dashboard."

18.2 More scenarios (give a step-by-step for each in an interview)

19. Deployment / DevOps

19.1 What's in the repo [ACTUAL IMPLEMENTATION]

19.2 Env vars

19.3 The serverless / scheduler mismatch (H-13)

19.4 How I'd deploy it properly [RECOMMENDED IMPROVEMENT]

20. Design decisions — "why did I choose this?"

D1. MongoDB (not Postgres)

D2. Modular monolith (not microservices)

D3. API key for the extension (not OAuth / JWT)

D4. Google OAuth → JWT in an httpOnly cookie (web)

D5. One `Activity` document per flush (not upsert/merge per day)

D6. Aggregate analytics in Node (not MongoDB `$group`)

D7. Deterministic statistics for Insights (not ML / not an LLM)

D8. `groupmembers` join table but `teams.members` embedded array

D9. React + Vite (not Next.js)

D10. 28 runtime themes via a Context + inline styles

21. Known limitations

21.1 Current limitations (this is a class/side project)

21.2 Production-level concerns (would block a real launch)

22. Future improvements roadmap

Short term (days)

Medium term (weeks)

Long term (months)

23. Time & space complexity

24. Code-level questions (with answers)

`middleware/auth.js` — `verifyApiKey`

`routes/leaderboard.js` — the aggregate

`routes/analytics.js` — `computeStreak`

`services/metricsService.js` — `buildMetrics`

`extension/src/editorTracker.ts` — `consumeChurn`

`extension/src/extension.ts` — `flushIfNeeded`

`frontend/src/pages/Dashboard.tsx` — the three fetches

`routes/goals.js` — `/:goalId/progress`

25. Codebase map

26. "Defend your project" — 30 interviewer exchanges

27. Final self-audit

Biggest interview risks to rehearse

1. Project overview & elevator pitches

What it is [ACTUAL IMPLEMENTATION]

CodeTrackr is a **developer-productivity analytics platform**. It has three parts that share one backend:

1. A **VS Code extension** that passively records coding activity (time, files, languages, editor edits, terminal commands, git commits, focus/flow) and POSTs it to the backend on a timer.
2. A **Node/Express/MongoDB backend** that authenticates the extension by an **API key**, normalises and stores each flush as an `Activity` document, and serves aggregated analytics to the dashboard.
3. A **React dashboard** that visualises a user's activity (daily/weekly charts, terminal analytics), plus social features — a **global leaderboard**, **study groups** with their own leaderboards, **goals** on a calendar with deadline notifications — and a private **Insights** page of derived productivity metrics (deep-work ratio, flow blocks, consistency, true peak hour, estimation accuracy).

Auth is split: the **web** uses Google OAuth → a JWT in an httpOnly cookie; the **extension** uses a per-user random API key in an `x-api-key` header.

The one honest caveat to lead with

The "ML/Insights" layer is **deterministic statistics**, not machine learning — coefficient of variation, weighted scoring, medians, computed on demand in pure JavaScript. I scoped it that way on purpose (explainable, reproducible, no training data needed). An LLM narration layer is *designed* in `docs/TRACKING_ROADMAP.md` but not built.

30-second version

"CodeTrackr is a coding-activity tracker. You install a VS Code extension, paste an API key from the website, and it quietly records what you work on — time per language, editor churn, terminal commands, git commits — and sends it to my backend every half minute or so. The React dashboard turns that into charts, a global leaderboard, study groups, goal tracking, and an insights page that tells you things like your most productive hour and how steady your habits are. It's a MERN stack — React, Express, MongoDB — plus a TypeScript VS Code extension."

1-minute version

"It solves a specific gap: most devs have no idea where their coding time actually goes. I built a VS Code extension in TypeScript that hooks into editor, terminal, git and window events and accumulates counters — gross lines edited, churn, focused minutes, flow blocks, command success rates. Every 30 seconds it flushes an activity document to an Express API, authenticated by a 64-character API key the user gets from the website after signing in with Google. The backend stores each flush in MongoDB and exposes aggregated endpoints. The React frontend renders daily and weekly charts with Chart.js, a leaderboard ranked by coding hours, groups you can create and join (public or password-protected), goals on a calendar with a node-cron job that fires deadline notifications, and an Insights page that derives five productivity metrics from the raw data using plain statistics. It's deployed with the frontend on Vercel, the backend on Render, and MongoDB Atlas."

2-minute version

Add, after the 1-minute version:

"The interesting design problem was linking the extension to the account without a full login flow inside VS Code. I went with an API-key model: on first Google sign-in the backend generates `crypto.randomBytes(32).hex` and stores it on the user; the extension sends it as a header; the backend does `User.findOne({ apiKey })` and attaches that user to the request. It's simple and it works, but in a review I'd call it what it is — an unscoped, non-expiring bearer credential stored in plaintext — and I can walk through how I'd harden it.

The other thing I'm proud of is the tracking accuracy work. The naive version measured time as wall-clock elapsed and lines as the net change in `document.LineCount`, which reports zero for any refactor that replaces code in place. I rewrote the trackers to accumulate gross insert/delete counts and a churn metric — lines you wrote then deleted within ten minutes — and to only count focused time, with 'flow blocks' that capture the shape of a session, not just its total. Those feed the Insights page.

I also did a security and correctness audit of my own code — there's an `IMPROVEMENT_PLAN.md` with 30-odd findings — and closed the high-severity ones: analytics endpoints were readable by user id with no ownership check, group passwords were plaintext, there was an `AUTH_BYPASS` env flag that disabled all auth. Those are fixed on the current branch with tests that statically scan the routes and fail if the auth middleware ever disappears."

5-minute deep technical version

Cover, in order: (1) the three-client / one-backend topology and why a modular monolith; (2) the extension's tracker architecture — `consumeInterval()` contract, idle/pause state machine, one Activity doc per flush, no offline queue; (3) the ingest path — `verifyApiKey`, additive normalisers that tolerate old payloads, `Activity.create`; (4) the read path — `isAuthenticated` + `assertOwnership`, `Activity.find` then JS aggregation, and why that's a known M-1 that should move into MongoDB `$group`; (5) the leaderboard's full-collection scan and the `UserStats` rollup you'd build instead; (6) the Insights pipeline — `metricsService`

(DB) + `metricsDerive` (pure, unit-tested) and the honest "this is stats not ML" line; (7) auth — Google OAuth → JWT cookie for web, API key for the extension, no refresh token, `session:false`; (8) the security findings you found and fixed vs the ones still open; (9) deployment — Vercel + Render + Atlas, and the `node-cron` -on-serverless problem; (10) what you'd do next — hash keys, add a queue and a rollup, adopt React Query, fix the 29 TypeScript errors, add integration tests.

"Tell me about your project."

"CodeTrackr is a coding-productivity tracker I built with two teammates. The core loop: a VS Code extension records your activity and sends it to my backend, which stores it in MongoDB, and a React dashboard visualises it with a leaderboard, groups and goals on top. My focus areas were the extension's activity-tracking accuracy, the backend API and data model, and a self-audit that turned into a security and correctness pass. The part I'd most want to talk through is the account-linking design — how the extension authenticates without a login UI — because it's a real trade-off and I know exactly where it's weak."

"What was your contribution?" [ground this ONLY in repo evidence]

The repo's git identity is **Soham Budhewar**; `extension/package.json` credits three contributors (Pratik Jambhule, Kartik Kharat, Soham). Honest phrasing:

"I worked across the stack. On the extension I built/rewrote the tracker layer — the editor, focus and git-state trackers with the `consumeInterval` reset contract, the idle-pause state machine, and the payload assembly — and fixed the packaging bug where `vsce` shipped stale code because `prepublish` never ran `tsc`. On the backend I built the metrics service and the derived-metrics module, the authorization and password-hash services, the activity normalisers, and the tests. I also did the codebase audit that produced `IMPROVEMENT_PLAN.md` and closed the high-severity security items. I can't claim the whole frontend — the dashboard and the decorative components predate my heaviest work — but I added the Insights page and the ownership checks that made the analytics endpoints safe to call with `credentials: 'include'`."

(Adjust to your real split. The safe move in an interview is to describe what you can explain in depth, not to claim ownership of code you'd struggle to defend.)

Why I built it

"I kept losing track of where my time went — I'd 'code for four hours' and have one commit to show for it. Existing tools like WakaTime measure time-in-editor but not whether that time was focused or churning. I wanted something that captured the shape of a session and gave me honest feedback, plus a social layer so a study group could see each other's consistency."

What problem it solves / what makes it different

- **Problem:** developers have poor visibility into how coding time is actually spent, and self-reported estimates are unreliable.
- **Different from a plain tracker:** (1) it measures **focused** time and **flow blocks**, not just elapsed time; (2) it tracks **churn** (write-then-delete) so a refactor isn't invisible; (3) it separates "busiest hour" from "most productive hour"; (4) it has a **group** layer for accountability; (5) it compares your goal **estimates** to hours you actually logged.

Hardest part / biggest technical challenge

"Getting the time number honest. The first version counted wall-clock elapsed from session start, so leaving VS Code open inflated everything. Then it counted lines as the delta of `document.LineCount`, which is zero for a replace-in-place edit — i.e. most refactoring. I ended up building a small state machine: mark activity on real editor/nav events, pause after two minutes idle, buffer active minutes, and flush with the timestamp of the start of the interval so hour-of-day analytics aren't skewed forward. The editor tracker accumulates gross insert/delete counts and attributes deletions against recent insertions to compute churn. All of that is unit-tested against the built bundle with a stubbed `vscode` module."

What I'd improve

Top three: (1) hash the API keys and support rotation with a grace window / per-device keys; (2) replace the leaderboard's full-collection scan with a `UserStats` rollup updated on ingest; (3) move analytics aggregation from JavaScript into MongoDB `$group` pipelines and add the missing `{userId:1, timestamp:-1}` index. Also: fix the 29 TypeScript errors so `npm run build` passes, and add integration tests that actually hit a database.

2. Architecture

(Diagrams and full data-flow walkthroughs are in `CodeTrackr_Architecture.md` . Summary here.)

Shape [ACTUAL IMPLEMENTATION]

- **Client-server**, three clients (extension, SPA, and — trivially — `curl`) against **one** Express backend and **one** MongoDB.
- **Modular monolith**: a single Node process; features are separate Express routers under `backend/routes/` . One deploy unit, one connection pool.
- **RESTish**: resource URLs + JSON + HTTP verbs; some RPC-ish paths (`/groups/:id/join`).
- **Partial MVC + thin service layer**: Mongoose models = M; routers = C; React = V. Services exist only for `metrics` , `authorization` , `passwordHash` , `activityNormalizers` , `notificationScheduler` . Everything else is route → Mongoose directly.
- **No** message queue, cache, gateway, search index, or WebSockets. NotificationPanel **polls** every 30 s.

Why this architecture

"It's a student/side project with one backend developer at a time and a free-tier budget. A monolith deploys as one Render service, debugs in one process, and has no cross-service consistency problems. The features genuinely share one data model — everything hangs off the `activities` and `users` collections — so splitting them into services would add network hops and distributed-transaction problems for zero benefit at this scale. The parts that would justify extraction later (high-volume ingest, heavy leaderboard aggregation) are exactly the parts I've identified for a queue + worker + rollup evolution."

Request lifecycles

Ingest: `POST /api/extension/track` → CORS (no Origin from a server-to-server call, so allowed) → `express.json()` → `verifyApiKey ( User.findOne({apiKey}) )` → handler validates `fileName/language/duration` → `normalize*Analytics()` → `Activity.create()` → `201` .

Dashboard read: `GET /api/analytics/:userId` → `express.json` / `cookie-parser` → `isAuthenticated` (`jwt.verify(req.cookies.token)` → `User.findById`) → `resolveOwnedUserId ( assertOwnership , 403 on mismatch)` → `Activity.find({userId, timestamp:{$gte:7d}})` → JS `.filter/.reduce/Map` → `res.json` .

Insights: `GET /api/metrics` → `isAuthenticated` → `buildMetrics(req.user._id)` (4 `Activity.aggregate` calls) → `metricsDerive` pure functions → `res.json` . **No :userId param at all** — identity comes only from the session, which makes this endpoint IDOR-proof by construction.

3. Technology stack (with alternatives & trade-offs)

For each: **what** / **where in CodeTrackr** / **why** / **alternative** / **why the choice holds** / **interview Qs.**

React 19 + Vite 7 + TypeScript

- **Where:** the entire dashboard SPA (`frontend/`). ~7,300 lines of TSX. Components, hooks (`useState` , `useEffect` , `useCallback` , `useRef` , `useContext`), one context (`ThemeContext`), `react-router-dom v7` for routing, prop-drilling for the `user` object.
- **Why:** component model fits a dashboard of many independent widgets; huge ecosystem; Vite gives sub-second HMR and a simple `vite build` ; TS catches shape errors on API responses.
- **Alternative:** Next.js (SSR/SEO, file routing, API routes co-located) — rejected because the app is authenticated and behind a login, so SEO is irrelevant and SSR adds a server to operate. Svelte/Vue — smaller bundles, but React's ecosystem (Chart.js wrapper, lucide icons) and team familiarity won.
- **Trade-off / where it bit me:** no data-fetching library — every navigation refetches with `cache: 'no-cache'` . `@tanstack/react-query` is even in `package.json` , unused (M-12). And `tsc -b` currently reports **29 errors** so `npm run build` fails (M-13).
- **Interview Qs:** "Why not Next.js?" / "How do you manage server state?" / "What breaks with StrictMode double-invoke?" / "Why is `useCallback` on `fetchAnalytics` ?"

Node.js 18 + Express 5

- **Where:** `backend/ . app.js` wires middleware and mounts 10 routers.
- **Why:** same language as the frontend and extension; Express is the smallest thing that does routing + middleware; Express 5 gives native async error propagation (a rejected promise in a handler becomes a 500 without `next(err)`) — though there's no central error handler so it's still a generic 500).
- **Alternative:** Fastify (faster, schema validation built in, would have covered M-4); NestJS (structure, DI, decorators — would have forced the service layer I only half-built); a Python/FastAPI backend if the ML plan were real.
- **Trade-off:** Express gives you nothing for free — no validation, no security headers, no rate limiting. `helmet` and `express-rate-limit` and `express-validator` are all installed and **not wired in** (M-3/M-4).
- **Interview Qs:** "Express 4 vs 5?" / "How does async error handling work here?" / "Where's your validation layer?"

MongoDB + Mongoose 8

- **Where:** the only datastore. `activities` (high volume, one doc per flush), `users` , `groups` , `groupmembers` , `goals` , `teams` , `notifications` .
- **Why:** activity documents are **append-only, self-contained, and schema-evolving** — I added three analytics sub-documents (`editorAnalytics` , `focusAnalytics` , `gitAnalytics`) additively with no migration, because Mongoose ignores unknown fields on read and defaults missing ones. The dominant access pattern is "all of one user's docs in a window, then aggregate" — no joins on the hot path. Mongoose gives schema validation and a familiar model API.

- **Alternative:** PostgreSQL — better for the *relational* parts (`groupmembers` , `teams.members` , goal ownership) where FKs and transactions matter, and dramatically better for the leaderboard, which is a `GROUP BY user_id` that Postgres does with a covering index instead of loading the whole table into Node. A time-series DB (Timescale, Influx) would fit the activity stream even better.
- **Trade-off:** no transactions anywhere; `activities.userId` is a `String` while everything else is `ObjectId` , forcing `$toObjectId` gymnastics and blocking `$lookup` (M-6); the leaderboard scan (H-7) is a direct consequence of "aggregate in the app, not the engine".
- **Interview Qs:** the whole of §26 exchanges 1–6.

VS Code Extension API (`@types/vscode ^1.85`) + `esbuild`

- **Where:** `extension/` . Events consumed: `workspace.onDidChangeTextDocument` , `onDidSaveTextDocument` , `window.onDidChangeActiveTextEditor` , `workspace.onDidOpenTextDocument` , `window.onDidChangeWindowState` , `window.onDidStart/EndTerminalShellExecution` , `debug.onDidStart/TerminateDebugSession` , `debug.onDidChangeBreakpoints` , and the built-in `vscode.git` extension's `repo.state.onDidChange` .
- **Why esbuild:** bundles `src/extension.ts` + all `src/*.ts` + `axios` into one `dist/extension.js` in milliseconds, `--external:vscode` (provided by the host). `vsce` packages that.
- **Alternative:** webpack (the VS Code default — slower, more config); `tsc` alone (no bundling, ships `node_modules`). `SecretStorage` instead of settings for the key (should have).
- **Trade-off:** `tsconfig` is `strict: false` (L-6); a stale `dist/` once shipped because `vscode:prepublish` ran `esbuild` but not `tsc` (fixed in 2.0.11).
- **Interview Qs:** §4 in full.

JSON Web Tokens (`jsonwebtoken`) + Passport (`passport-google-oauth20`) + `cookie-parser`

- **Where:** `routes/auth.js` signs the JWT on OAuth callback; `middleware/auth.js` verifies it from the `token` cookie. `config/passport.js` is the Google strategy.
- **Why:** OAuth means no password storage for users; a stateless JWT in an `httpOnly` cookie means no session table and it survives a serverless cold start.
- **Alternative:** server-side sessions (`express-session` + a store) — easier revocation, but needs shared state; Auth0/Clerk — less code, external dependency.
- **Trade-off:** `JWT_SECRET` has a hardcoded fallback `'your_jwt_secret'` (M-9); no refresh token, so a 1-day expiry is a hard logout; no server-side revocation; `passport.session()` isn't used so `serializeUser` / `deserializeUser` are dead code.
- **Interview Qs:** §26 exchanges on JWT vs sessions, refresh tokens, CSRF.

`crypto` (Node built-in) — API keys & group passwords

- **Where:** `user.generateApiKey() = randomBytes(32).toString('hex')` ; `services/passwordHash.js = crypto.scrypt` for group passwords (`scrypt$salt$hash`).
- **Why scrypt over bcrypt:** no native addon → the Vercel serverless build stays simple and the tests run without `npm install` .
- **Alternative:** `bcrypt` / `argon2` (native, battle-tested); a KMS for the API keys.
- **Trade-off:** the **API key itself is still plaintext at rest** — only *group* passwords are hashed.

MongoDB driver via Mongoose — no ORM/query builder beyond it. `node-cron` for scheduling.

- `node-cron` : `'0 * * * *'` hourly + immediate run. Fine on a long-lived Render process; **broken on serverless** (H-13).
- **Alternative:** Vercel Cron, a GitHub Action on a schedule, BullMQ repeatable jobs, or Atlas Triggers.

Chart.js 4 + react-chartjs-2 + chartjs-plugin-datalabels

- **Where:** `Dashboard.tsx` — Line (hourly/daily hours), Pie (language breakdown), Bar (terminal success/failure, command usage, build/git trends).
- **Alternative:** Recharts (React-native components, declarative), visx/D3 (control, more code), Nivo.
- **Trade-off:** Chart.js is canvas — not accessible, not easily server-rendered, and you register scales/elements globally. Recharts would have been more idiomatic React.

Tailwind CSS 3 + a bespoke `ThemeContext`

- **Where:** utility classes for layout; **inline** `style={{}}` **with** `theme.colors.*` for anything colour-related, because there are **28 runtime-switchable themes** stored in `localStorage` and applied as CSS custom properties.
- **Trade-off:** the inline-style + theme-object pattern is verbose and repeated in every component; a proper design-token system (CSS variables + Tailwind `theme.extend`) would cut it drastically.
`tailwind.config.js` `theme.extend` is empty.

Deployment: Vercel (frontend) + Render (backend) + MongoDB Atlas

- **Why:** all have free tiers; Vercel is one-click for a Vite SPA (`vercel.json` SPA rewrite); Render runs a plain Node process so `app.listen()` and `node-cron` work; Atlas is managed Mongo.
- **Trade-off:** `backend/vercel.json` exists too (serverless via `serverless-http`) — if that's the live deploy, the scheduler is broken. Render free tier sleeps after 15 min idle → cold-start latency and a missed cron window.

Not used (and you'll be asked why): Redis, Kafka/SQS, WebSockets, GraphQL, Docker, any CI.

See §26 for the rebuttals.

4. The VS Code extension — in depth

Files: `extension/src/*.ts` . Entry `extension.ts` . Built to `dist/extension.js` by `esbuild`.

4.1 Installation → activation

- Published as `CodeTrackr-ext.codetracker-vscode` . `main: ./dist/extension.js` .
- **Activation event:** `onStartupFinished` (activates shortly after VS Code finishes starting — not lazy on a command, because tracking should be passive).
- `activate(context)` :
 1. `initializeTelemetry(context)` — snapshot terminal count, subscribe to open/close.
 2. Create the five trackers (`createDebugTracker` , `createTerminalTracker` , `createEditorTracker` , `createFocusTracker` , `createGitStateTracker`) and `.start(context)` each.
 3. Register 7 commands: `start` , `stop` , `flushNow` , `showStats` , `showLogs` , `setupApiKey` , `showInfo` .
 4. Subscribe to `onDidChangeConfiguration` — restart the flush timer if `flushIntervalSeconds` changes; clear the auth warning if `apiKey` changes.
 5. `start(context)` — begin tracking.
 6. If no API key configured → one-time actionable warning.
- `activation.test.js` loads the real built bundle against a stubbed `vscode` and asserts every contributed command is registered, no network call happens without a key, and the payload has the four analytics blocks.

4.2 The flush loop and idle/pause state machine

State (`AppState`): `timer` , `lastActivityMs` , `startedMs` , `bufferedMinutes` , `lastKnownFile` , `activeTerminalCount` , `isPaused` .

`start()` subscribes to `onDidOpenTextDocument` , `onDidSaveTextDocument` , `onDidChangeActiveTextEditor` , `onDidChangeTextDocument` — all call `markActivity()` which sets `lastActivityMs = Date.now()` and, if paused, resumes (new `startedMs` , clears buffer).

`setInterval(flushIntervalSeconds * 1000)` (default **30 s**) tick:

```
idleMin = (now - lastActivityMs) / 60000
if (!isPaused && idleMin >= 2):
    if there was active time: sendActivity(activeDurationMin, lastKnownFile) // flush the tail
    isPaused = true ; startedMs = undefined ; bufferedMinutes = 0
    return
if (isPaused): return
flushIfNeeded(false)
```

`flushIfNeeded(force)` :


```

if (!startedMs) return
elapsedFromStartMin = (now - startedMs)/60000
idleMin = (now - lastActivityMs)/60000
if (!force && idleMin >= 2) return // idle: don't flush
totalBuffered = bufferedMinutes + elapsedFromStartMin
if (!force && totalBuffered < minFlushMinutes) return // default 0.5 min
await sendActivity(totalBuffered, fileOpened)
startedMs = now ; bufferedMinutes = 0 // on success
// on throw: bufferedMinutes = totalBuffered ; startedMs = now (retry next tick)

```

4.3 What is collected, and how

Signal	Source	Tracker	Notes
Active time	timer + <code>markActivity()</code> events	<code>extension.ts</code>	buffered minutes; idle < 2 min still counts (L-7)
File name / type / project / language	<code>activeTextEditor.document</code> , <code>workspace.name</code>	<code>getFileMeta()</code>	basename only — never the path
Gross lines/chars inserted & deleted	<code>onDidChangeTextDocument</code> content changes	<code>EditorTracker</code>	replaces the broken <code>net-lineCount</code> delta
Churn	deletions matched against insertions ≤ 10 min old	<code>EditorTracker.consumeChurn</code>	"wrote it then deleted it"
Undo/redo	change <code>reason === 1 / 2</code>	<code>EditorTracker</code>	
Saves, file switches, unique files	save + active-editor events	<code>EditorTracker</code>	
Read vs write time	5 s sampler: was there an edit in the last interval?	<code>EditorTracker.sampleAttention</code>	only samples while <code>window.state.focused</code>
Large inserts	insert text length > 80 chars	<code>EditorTracker</code>	count + total chars, never the text
Focused / blurred ms, blur events	<code>window.onDidChangeWindowState</code>	<code>FocusTracker</code>	removes "VS Code open but not foreground" time
Flow blocks	working stretch; closes after 2 min idle	<code>FocusTracker.tick</code>	<code>flowBlocksMs[]</code> + <code>longestBlockMs</code> ; open block left open across flushes
Commits, files changed, uncommitted count/age	<code>vscode.git</code> extension API, <code>repo.state.HEAD.commit</code>	<code>GitStateTracker</code>	counts GUI commits too; never messages/diffs
Terminal commands: total, success/fail, per-category, build/test runs, git actions, repeated failures	<code>onDidStart/EndTerminalShellExecution</code>	<code>TerminalTracker</code> + <code>analyticsAggregator</code> + <code>commandClassifier</code>	exit code → success/fail; commands sanitised (<code>token=***</code>) + truncated
Debug sessions	<code>debug.onDidStart/TerminateDebugSession</code>	<code>DebugTracker</code>	folded into <code>terminalAnalytics.debuggingSessions</code>

4.4 Frequency, batching, buffering, debounce/throttle

- **Timer ticks** every `flushIntervalSeconds` (default 30 s).

- **A flush happens** only when `totalBuffered ≥ minFlushMinutes` (default 0.5 min) — so in practice roughly every 30–90 s of continuous coding.
- **Batched?** No. One `POST /api/extension/track` per flush, one `Activity` doc per POST. A `/track/batch` endpoint exists on the backend but the extension never calls it.
- **Buffering:** active minutes are buffered in the `state` object between ticks and across a failed flush (in memory only).
- **Debounce/throttle:** the 5 s attention sampler and the 15 s focus ticker are the only fixed intervals; edits themselves aren't debounced — they just bump `lastActivityMs` and increment counters.

4.5 Lifecycle edge cases [ACTUAL IMPLEMENTATION]

Situation	Behaviour
User idle > 2 min	Flush the tail, <code>isPaused = true</code> , stop buffering. Resume on next edit/nav.
VS Code closes	<code>deactivate()</code> does a best-effort final <code>flushIfNeeded(true)</code> , then stops trackers. If that POST fails, the tail is lost.
No internet / backend down	<code>axios</code> throws → buffered minutes retained in memory ; next tick retries. No disk queue — a restart loses everything un-flushed.
Invalid / rotated key	<code>verify</code> or a flush returns 401 → one-time <code>showWarningMessage</code> with "Set API Key" / "Open Dashboard". <code>authWarningShown</code> gates it to once per session.
Key "expires"	Keys don't expire. Only regeneration invalidates one.
User switches account	They regenerate on the new account and re-run Setup API Key; the old key stops working. The extension has no concept of "which account" beyond the key string.
Duplicate activity / same payload twice	The backend stores it again — no idempotency . Double flush = double count.
Clock jump	<code>duration > 3600 s</code> → the flush is skipped with a console warning. <code>duration < 1 s</code> → held for the next flush (backend 400s on falsy duration).
<code>duration === 0</code>	Backend <code>if (!duration)</code> wrongly rejects it as missing (M-4). The extension avoids sending it.

4.6 Where the API key lives locally

`vscode.workspace.getConfiguration('codetrackr').update('apiKey', key, vscode.ConfigurationTarget.Global)` → **plaintext in the user's global** `settings.json`. **Not** VS Code SecretStorage. `showInfo` masks it (`abcd...wxyz` (64 chars)). **[RECOMMENDED IMPROVEMENT]** move to `context.secrets` (SecretStorage), which uses the OS keychain.

4.7 Source-file map (extension)

File	Key exports / functions
<code>extension.ts</code>	<code>activate</code> , <code>deactivate</code> , <code>buildPayload</code> , <code>buildPayloadForTest</code> , <code>sendActivity</code> , <code>flushIfNeeded</code> , <code>start</code> , <code>stop</code> , <code>setupApiKey</code> , <code>showInfo</code> , <code>showStats</code>
<code>editorTracker.ts</code>	<code>EditorTracker</code> (<code>recordChange</code> , <code>consumeChurn</code> , <code>sampleAttention</code> , <code>consumeInterval</code>), <code>createEditorTracker</code>
<code>focusTracker.ts</code>	<code>FocusTracker</code> (<code>setFocused</code> , <code>noteActivity</code> , <code>tick</code> , <code>accrue</code> , <code>consumeInterval</code>), <code>createFocusTracker</code>
<code>gitStateTracker.ts</code>	<code>GitStateTracker</code> (<code>handleStateChange</code> , <code>consumeInterval</code>), <code>createGitStateTracker</code>

File	Key exports / functions
terminalTracker.ts	TerminalTracker (onExecutionStart/End , consumeInterval), createTerminalTracker
analyticsAggregator.ts	TerminalAnalyticsAggregator (recordExecution , collectRepeatedFailures , consumeInterval)
commandClassifier.ts	classifyCommand , normalizeCommand , sanitizeCommand
gitTracker.ts	detectGitAction
debugTracker.ts	DebugTracker (onSessionStart/End , onBreakpointChange , consumeInterval)

5. The unique-key / account-linking system

5.1 End to end [ACTUAL IMPLEMENTATION]

Step	Code	Detail
Generate	<code>config/passport.js</code> → <code>user.generateApiKey()</code> (<code>models/user.js</code>)	On first Google login. <code>crypto.randomBytes(32).toString('hex')</code> → 64 hex chars (256 bits of entropy).
Store	<code>models/user.js</code>	<code>apiKey</code> : { type: String, unique: true, sparse: true } — plaintext , unique index, <code>sparse</code> so users created before a key don't collide on <code>null</code> .
Show to user	<code>routes/user.js</code> GET <code>/api/user/profile</code> → <code>Onboarding.tsx</code> / <code>Profile.tsx</code>	Returned in the JSON body; copy-to-clipboard button.
Extension receives	<code>extension.ts</code> <code>setupApiKey()</code>	<code>showInputBox({ password:true })</code> , <code>>= 16</code> chars validation, saved to global config.
Extension sends	<code>extension.ts</code> <code>sendActivity()</code>	<code>headers</code> : { 'x-api-key': <code>apiKey</code> } on every POST <code>/api/extension/track</code> .
Backend verifies	<code>middleware/auth.js</code> <code>verifyApiKey</code>	<code>apiKey = req.headers['x-api-key'].trim()</code> ; <code>User.findOne({ apiKey })</code> ; <code>!user</code> → 401; else <code>req.user = user</code> .
Attribution	<code>routes/extension.js</code>	<code>Activity.userId = req.user._id.toString()</code> .
Rotate	<code>routes/user.js</code> POST <code>/api/user/regenerate-api-key</code>	Overwrites <code>apiKey</code> ; old key dead immediately.

Uniqueness guarantee: 256-bit random + a unique index. Collision probability is cryptographically negligible; the index would reject a dup on write anyway.

5.2 Is this actually "just an identifier"? — the discussion an interviewer wants

No. Distinguish the roles:

Role	Does the key play it?	Why it matters
Identifier	Yes — resolves to exactly one user.	A pure identifier (like a username or a public user id) is <i>safe to expose</i> .
Authenticator	Yes — possession alone authorises writes; there's no second factor.	This is what makes it a <i>credential</i> , not an id.
Bearer token	Yes — sent on every request, no proof-of-possession, no signature.	Anyone who observes it can replay it.
Pairing token	Yes — binds one extension install to one account.	But it never "completes" a pairing; it stays live forever.
Secret	It's <i>treated</i> as one (masked in UI, <code>password:true</code> input) but stored and compared in plaintext .	If the DB leaks, every key leaks in usable form.
Scoped	No — full ingest authority, no per-resource scope.	Can't issue a "write-only, this-device" key.
Expiring	No .	A leaked key is valid until the user manually rotates.

Verdict to say out loud: "It's functionally an unscoped, non-expiring API token stored in plaintext. That's a defensible MVP choice — it's one `findOne`, no token minting, works offline — but I'd call it a credential in a review, not an identifier, and I know how I'd harden it."

5.3 Attack scenarios [ACTUAL IMPLEMENTATION risk]

1. **Key theft → leaderboard fraud.** With a stolen key, `curl POST /api/extension/track` with `duration: 3600`, `language: "TypeScript"` in a loop → the victim's coding hours (and the global leaderboard) are corrupted. No rate limit, no anomaly check, no idempotency. *Bounded by:* the attacker still can't read the victim's dashboard (needs the JWT).
2. **DB compromise → mass key leak.** Keys are plaintext, so a read of `users` yields working credentials for everyone.
3. **Accidental exposure.** The key is shown in the dashboard and pasted into `settings.json`, which people commit to dotfiles repos. It's also in `GET /api/user/profile` responses in browser history / logs.
4. **No brute force needed** — 256 bits — but also **no lockout** if someone tried.
5. **Self-inflicted inflation.** A user can point their own key at `curl` and cheat the leaderboard directly. Cheating is trivial and undetected (see §26).

5.4 How to redesign it [RECOMMENDED IMPROVEMENT]

- **Hash at rest.** Store `sha256(key)` (fast, keys are high-entropy so a slow KDF isn't required) or `srypt`. Issue as `ct_<keyId>_<secret>`; look up by indexed `keyId`, compare the secret in constant time. DB leak no longer yields usable keys.
- **Multiple named keys per user**, each independently revocable ("Work laptop", "Desktop").
- **Rotation with a grace window** — old key valid for 24 h after rotation so tracking doesn't silently break.
- **Scope + metadata** — `scope: ['ingest']`, `lastUsedAt`, `createdAt`, `userAgent`.
- **Better still:** the dashboard mints a **short-lived signed ingest token** (JWT, 24 h, `aud: 'ingest'`) that the extension refreshes; or an **OAuth 2.0 device-authorization flow** so the extension never handles a long-lived secret.
- **Defence in depth on ingest:** rate-limit per key, reject implausible `duration`, require a client-generated idempotency key, flag statistical anomalies.

6. Database design (MongoDB / Mongoose)

6.1 Collections, purpose, structure

activities — the only high-volume collection

```
{
  userId: String,           // hex of users._id — NOT an ObjectId ref
  fileName: String,        // required, basename only
  fileType, projectName, language: String,
  duration: Number,        // SECONDS ← every reader divides by 3600
  linesAdded, linesRemoved: Number, // gross counts since 2.2.0
  timestamp: Date,         // start of the flush interval
  date: Date,              // == timestamp since H-5; legacy rows differ
  terminalAnalytics: { totalCommands, successfulCommands, failedCommands, successRate,
                      buildRuns, testRuns, successfulBuilds, failedBuilds, buildSuccessRate,
                      debuggingSessions, commandUsage: {git, npm, node, python, docker, gcc, java, pip, misc},
                      gitActivity: {commits, pushes, pulls, checkouts, merges, clones},
                      repeatedFailedCommands: [{command, count}], lastCommand, lastCommandTimestamp },
  editorAnalytics: { charsInserted, charsDeleted, linesInserted, linesDeleted, churnLines,
                    undoCount, redoCount, saveCount, fileSwitches, uniqueFiles,
                    readMs, writeMs, largeInsertCount, largeInsertChars },
  focusAnalytics: { focusedMs, blurredMs, blurEvents, flowBlocksMs: [Number], longestBlockMs },
  gitAnalytics: { commits, filesChanged, uncommittedFiles, uncommittedAgeMs }
} // { timestamps: true } adds createdAt / updatedAt
```

- **Purpose:** immutable record of one flush interval.
- **Relationships:** `userId` → `users._id` (as a string; no populate).
- **Why this structure:** self-contained, append-only, cheap to write; sub-documents added additively without migration.
- **Scalability issue:** one doc per flush → ~ 60–120 docs/user/active-hour; unbounded growth; all analytics scan a user's slice and aggregate in Node.
- **Improvements:** TTL or cold-storage archival of raw docs after N months; a `UserStats` daily rollout; migrate `userId` to `ObjectId`; add `{userId:1,timestamp:-1}`.

users

`googleId` (unique), `email` (unique), `name`, `profilePictureUrl`, `apiKey` (unique, sparse, plaintext), `lastLogin`, `isFirstLogin`. Purpose: identity + the ingest credential.

groups / groupmembers

`groups`: `name`, `description`, `visibility`, `password` (`select:false`, script hash), `createdBy` (`ObjectId`→`User`). `groupmembers`: { `groupId`, `userId`, `joinedAt` } with **unique compound index** `{groupId:1,userId:1}` — a real join table. This is the correct many-to-many modelling and the compound unique index is the one hard concurrency guard.

goals

`userId` (`ObjectId`), `title`, `description`, `targetHours` (min 1), `techStack` (String), `deadline`, `status` (`in-progress` / `completed`, default `in-progress`), `reminderSent`. **No route sets** `status: 'completed'` — only the seed script does. Progress is computed on demand, not stored.

teams

`name`, `description`, `createdBy`, `members: [ObjectId]` **embedded**. Backend routes exist; **no frontend route** — orphaned. A deliberately different modelling choice from groups (embedded array vs join table) — good talking point on embedding vs referencing.

notifications

`userId`, `goalId`, `type` (enum incl. `goal_completed` which is never produced), `title`, `message`, `read`.
Written by `notificationScheduler`.

6.2 Indexes [ACTUAL IMPLEMENTATION + gap]

Defined: `{userId:1}`, `{userId:1,date:-1}`, `{userId:1,projectName:1}`, `{userId:1,language:1}` on `activities`; `{groupId:1,userId:1}` unique on `groupmembers`; unique on `users.googleId`, `users.email`, `users.apikey` (sparse).

Missing / wrong:

- No `{userId:1,timestamp:-1}` — but every analytics/metrics/streak query filters `timestamp`. They use the `{userId:1}` index and filter timestamp in memory.
- The `{userId:1,date:-1}` index is near-useless because nothing queries `date`.
- No index supporting the leaderboard's collection-wide `$group` (there's no good one — it's a full scan by nature).

6.3 SQL vs MongoDB — the full answer

Why Mongo fits here: append-only documents, schema evolution without migrations, per-user-window access pattern, no hot-path joins, free-tier managed hosting (Atlas), team familiarity, and a single model file per entity.

When Postgres would be better:

- **Relational integrity:** `groupmembers`, `teams.members`, goal/notification ownership — foreign keys and `ON DELETE CASCADE` would remove the "filter out null groups if deleted" defensive code in `groups.js`.
- **The leaderboard:** `SELECT user_id, SUM(duration) ... GROUP BY user_id ORDER BY 2 DESC LIMIT 50` with a covering index is O(index scan), not "load the whole table into Node".
- **Transactions:** `team.members.push()` + `save()` is a read-modify-write with a lost-update window; SQL `INSERT ... ON CONFLICT DO NOTHING` on a join table is atomic.
- **Ad-hoc analytics:** window functions, `GROUP BY GROUPING SETS`, percentiles.

When a time-series DB (Timescale/Influx) would be better still: the activity stream is literally time-series — automatic partitioning by time, continuous aggregates (materialised rollups), retention policies. That's the "right" store for `activities` at scale.

6.4 Consistency, races, atomicity

- **No transactions.** Every write is a single document op, which Mongo makes atomic — so ingest is fine.

- **Double-join race:** two concurrent `POST /:groupId/join` → the second hits the unique compound index → duplicate-key error, currently surfaced as a **500** (should be a 409).
- **Last-member-leaves race:** two members leave simultaneously; both see `remainingMembers` briefly and one deletes the group — benign but sloppy; a transaction or a `findOneAndDelete` guard would tidy it.
- `team.members.push` **lost update:** concurrent adds can clobber each other. Fix: `$addToSet`.
- **Leaderboard:** recomputed per request, so always consistent, never stale, never cached.

7. Backend & API architecture

7.1 Endpoint table

Method	Endpoint	Purpose	Input	Auth	Processing	Response
POST	/api/extension/track	ingest one flush	body: fileName, language, duration(s), timestamp?, analytics sub-objects	verifyApiKey (x-api-key)	validate → normalise → Activity.create	201 {success, activity: {id,...}}
POST	/api/extension/track/batch	ingest many	{ activities: [...] }	verifyApiKey	map+normalise → Activity.insert Many	201 {success, count}
GET	/api/extension/verify	key check	—	verifyApiKey	—	200 {success, user: {id,name,email}}
GET	/api/analytics/:userId?timezone=	today hourly + 7d totals	tz offset (min)	isAuthenticated + ownership	Activity.find(7d) → JS reduce; computeStreak (\$group)	{ totalHours, projectCount, totalLinesAdded, streakDays, dailyActivity[24], languageBreakdown[], terminalSummary, *Timeline[] }
GET	/api/analytics/weekly/:userId?timezone=	7-day daily	tz	isAuthenticated + ownership	Activity.find(7d) → JS reduce	daily breakdown + language + terminal
GET	/api/analytics/timeslot/:userId?start=&end=&timezone=	2h drill-down	start/end hour, tz	isAuthenticated + ownership	Activity.find(window) → 10-min slots	{ totalMinutes, totalLines, fileCount, productivity, tenMinuteSlots[12], languages[] }
GET	/api/analytics/summary/:userId	daily + per-stack totals	—	isAuthenticated + ownership	2× Activity.aggregate \$group	{ dailyTotals[], stackTotals[] }
GET	/api/leaderboard	global ranking	—	isAuthenticated	User.find({}) + Activity.aggregate(all) → merge/sort/score in Node	[{ rank, userId, name, email, totalHours, ... , speed, quality, engagement, impact, overall }]
GET	/api/metrics?days=&timezone=	derived insights	days (1–365), tz	isAuthenticated (no :userId)	4× Activity.aggregate → metricsDerive	{ success, metrics:{...11 fields...} }
POST	/api/goals/create	new goal	title, description, targetHours, techStack, deadline	isAuthenticated	Goal.create (userId from session)	201 goal
GET	/api/goals	my goals	—	isAuthenticated	Goal.find({userId})	[goal]
GET	/api/goals/:goalId/progress	goal progress	—	isAuthenticated + owner scope	Goal.findOne({_id,userId}) + Activity.find → reduce	{ goal, currentHours, progress }

Method	Endpoint	Purpose	Input	Auth	Processing	Response
POST	/api/groups/create	new group	groupName, groupDescription, visibility, password?	isAuthenticated	hash pw → Group.save → GroupMember.save (creator)	201 {group} (no password)
GET	/api/groups/my-groups	member of	—	isAuthenticated	GroupMember.find({userId}).populate(groupId)	{groups:[]}
GET	/api/groups/discover?search=	not a member of	search?	isAuthenticated	Group.find({_id: {\$nin:joined}}) + regex	{groups:[]}
GET	/api/groups/:groupId/details	members + leaderboard	—	isAuthenticated + membersh ip	GroupMember.find().populate + Activity.aggregate({\$in:memberIds})	{group, members[], leaderboard[]}
POST	/api/groups/:groupId/join	join	password?	isAuthenticated	verify pw (private) → GroupMember.save	{success, group}
POST	/api/groups/:groupId/leave	leave	—	isAuthenticated	GroupMember.findOneAndDelete; delete group if empty	{success, message}
GET	/api/notifications · /unread-count	list / count	—	isAuthenticated	Notification.find/countDocuments({userId})	list (≤50) / {count}
PATCH	/api/notifications/:id/read · /mark-all-read	mark read	—	isAuthenticated	findOneAndUpdate({_id,userId}) / updateMany	{notification} / {message}
DELETE	/api/notifications/:id	delete	—	isAuthenticated	findOneAndDelete({_id,userId})	{message}
POST	/api/teams/create · GET /api/teams · GET /api/teams/:teamId · POST /api/teams/:teamId/members	teams	—	isAuthenticated (+ membership / admin)	Team CRUD	team objects — no UI
GET	/auth/google · /auth/google/callback	OAuth	—	—	passport → sign JWT → cookie → redirect	redirect
GET	/auth/current-user	decode cookie	—	—	jwt.verify	{user} / 401
POST	/auth/logout	clear cookie	—	—	res.clearCookie	{success}
GET	/api/user/profile	my profile + apiKey	—	isAuthenticated	User.findById	{success, user:{..., apiKey}}

Method	Endpoint	Purpose	Input	Auth	Processing	Response
POST	/api/user/regenerate-api-key	rotate key	—	isAuthenticated	generateApiKey + save	{success, apiKey}
POST	/api/user/complete-onboarding	flip flag	—	isAuthenticated	isFirstLogin = false	{success}

7.2 Middleware pipeline

app.js: cors(allowlist, credentials:true) → express.json() → cookieParser() → passport.initialize() → routers. **No** helmet, **no** rate limiter, **no** validator, **no** central error handler. Each route wraps its body in try/catch and returns 500 { message, error: err.message } (leaks internals — M-10).

7.3 Status codes actually used

200 reads, 201 creates, 400 missing fields / bad body, 401 unauth/invalid-key/expired-JWT, 403 ownership/membership failure, 404 not found, 500 anything thrown. No 409 (double-join returns 500), no 422, no 429 (no rate limiting).

7.4 API design weaknesses

- GET endpoints still take :userId in the path even though it's verified against the session — kept for "one release of backward compatibility" (H-1 note); cleaner to drop it.
- POST /api/user/regenerate-api-key and complete-onboarding are POST with no body and no CSRF token.
- Error bodies echo err.message.
- No pagination anywhere (notifications is hard-capped at 50; leaderboard/groups unbounded).
- No API versioning.
- /track uses !duration so 0 is rejected.

8. Frontend architecture

8.1 Structure

- `main.tsx` → `<StrictMode><ThemeProvider><App/></ThemeProvider>`.
- `App.tsx` — `BrowserRouter`, auth gate (`GET /api/user/profile` on mount), nav shell, `<Routes>`. Unauthenticated users get only `/login`.
- Pages: `Login`, `Onboarding`, `Dashboard`, `Insights`, `Leaderboard`, `Goals`, `Groups`, `Profile`. (`Teams` exists, is not routed.)
- Components: `NotificationPanel`, `ThemeSelector`, `GradientText`, `TextType`, `ElectricBorder`, and heavy decorative WebGL/canvas pieces (`Orb` via OGL, `Hyperspeed`, `LetterGlitch`, `TargetCursor`).

8.2 State management [ACTUAL IMPLEMENTATION]

- No data-state library.** `useState` per page + **prop-drilling** the `user` object into `Dashboard` and `Groups`.
- The only Context** is `ThemeContext` (theme name + palette + `setTheme`, persisted to `localStorage`).
- Auth state** lives in `App.tsx` (`user` / `loading`), derived from one profile fetch.
- Server state** is refetched on every navigation with `cache:'no-cache'` + `Pragma` headers. `@tanstack/react-query` is a dependency and **completely unused** (M-12).
- Local UI state** examples: `Dashboard` `viewMode`, `selectedTimeSlot`; `Groups` `activeTab`, modal flags; `Insights` `days`.

8.3 React concepts actually used (and the questions they invite)

Concept	Where	Interview Q
<code>useEffect([])</code> for data on mount	every page	"What runs twice in StrictMode and does it matter here?" (fetches — idempotent GETs, so fine)
<code>useEffect([dep])</code> refetch	<code>Dashboard</code> <code>[viewMode]</code> , <code>Insights</code> <code>[fetchMetrics]</code>	" <code>Dashboard</code> fires 3 requests on mount, one redundant — why?" (M-11: <code>[]</code> + <code>[viewMode]</code> both call <code>fetchAnalytics</code>)
<code>useCallback</code>	<code>fetchAnalytics</code> , <code>fetchMetrics</code>	"Why memoise these?" (stable identity for the effect dep)
<code>useRef</code>	<code>NotificationPanel</code> <code>panelRef</code> (click-outside)	
<code>useContext</code> custom hook	<code>useTheme()</code> throws if outside provider	"Why throw?" (fail fast on a wiring bug)
Conditional rendering	<code>loading</code> / <code>error</code> / <code>empty</code> states everywhere	
Lists + keys	<code>leaderboard</code> rows keyed by <code>userId</code> , <code>groups</code> by <code>_id</code>	
Controlled inputs	every form (<code>Goals</code> , <code>Groups</code> , <code>Teams</code>)	
Prop-drilling	<code>user</code> → <code>Dashboard/Groups</code>	"When would you lift this to Context / a store?"
Derived state in render	<code>Leaderboard</code> team average, <code>Dashboard</code> chart datasets	"Should these be <code>useMemo</code> ?" (cheap enough not to)

8.4 Data flow (frontend)



No API client abstraction — each component calls `fetch(\ ${API_URL}/...`, {credentials:'include'})` directly. `API_URL` from `config.ts``.

8.5 Loading / error / empty states

Consistently handled: each page has a `loading` boolean → spinner text; failed fetches `console.error` and often leave the page in its empty state; `Insights` has an explicit error card with "Try again"; `Leaderboard / Groups / Dashboard` render "No data yet" empties. `Groups` uses `alert()` for errors (crude but present).

8.6 Known frontend problems [ACTUAL IMPLEMENTATION]

1. `npm run build fails` — `tsc -b` → 29 errors (16 in `Goals.tsx`, 5 `TextType.tsx`, 3 each `Groups / Dashboard`, 1 each `Teams / Profile`); 2 are real (`string not assignable to never`), the rest unused imports/vars (M-13).
2. **Dashboard "Repeated Failures"** renders `repeatedFailuresDaily / repeatedFailuresWeekly` — **hardcoded arrays** — instead of `terminalSummary.repeatedFailedCommands`, which the backend does compute.
3. `Goals.tsx to-dos` (`addTodo / toggleTodo / deleteTodo`) mutate React state only — never persisted, no API. No goal completion / delete / progress-bar UI; the `/:goalId/progress` endpoint is never called.
4. `NotificationPanel` polling `useEffect` closes over a stale `isOpen` (L-1) — the "refetch full list while open" branch never runs from the interval.
5. `Teams.tsx` is dead (no route).
6. No client caching / dedup — every nav refetches.

9. Analytics processing

9.1 Where it happens [ACTUAL IMPLEMENTATION]

Almost entirely **in Node, not MongoDB** (`routes/analytics.js`): `Activity.find(...)` then `.filter / .reduce / Map`. Exceptions: `computeStreak` and `/summary/:userId` use real `$group` pipelines; `/api/metrics` uses `$group` throughout.

9.2 The daily endpoint, step by step

1. `Activity.find({ userId, timestamp: { $gte: sevenDaysAgo } })` — **7 days of docs to render 1 day** (M-1).
2. Compute user-local midnight from the `?timezone=` offset (minutes, `Date.getTimezoneOffset()` semantics).
3. `todayActivities = activities.filter(in [startOfToday, endOfToday])`.
4. Totals: $\Sigma \text{duration} / 3600 \rightarrow \text{hours}$; `new Set(projectName)` $\rightarrow$ project count; $\Sigma \text{linesAdded}$.
5. `computeStreak(userId, tzOffset)` — separate 90-day `$group` by local-day string, Set membership, walk backward from today (or yesterday, so a not-yet-started day doesn't break the streak).
6. Hourly map (24 buckets) of hours; language map $\rightarrow$ sorted breakdown.
7. `buildTerminalSummary(todayActivities)` — sum every terminal counter, merge `commandUsage`, top-5 repeated failures, recompute `successRate` / `buildSuccessRate`.
8. Hourly terminal / build / git timelines (24 buckets each).

9.3 Timezone handling

The client sends `new Date().getTimezoneOffset()` (minutes, **negative east of UTC**) as `?timezone=`. `localDayInfo(instant, offset)` subtracts `offset*60000` and takes the ISO date. The weekly endpoint historically **ignored** the offset (M-2) — now fixed.

9.4 Correctness history (all in `IMPROVEMENT_PLAN.md`, all fixed)

- **H-3:** goal progress was 60 $\times$ overstated — `duration` (seconds) was treated as minutes.
- **H-4:** `/summary/:userId` always returned zero (bad `$match` type).
- **H-5:** `date` was written as the ingest day, so backdated flushes landed on "today".
- **H-6:** streak was wrong three ways — seeded to 1 with no recency check, capped at 7 days, no timezone bucketing.

9.5 Complexity

For a user with N activity docs in the window: `find` is $O(N)$ docs transferred + parsed; JS aggregation is $O(N)$; memory $O(N)$ (all docs resident). Streak: $O(N_{90})$ where N_{90} is 90-day docs. The dominant cost is **transferring and deserialising every document** rather than letting Mongo aggregate — which is exactly why M-1 matters at scale.

10. Leaderboard

10.1 What determines rank [ACTUAL IMPLEMENTATION]

`totalHours =  $\Sigma$  duration / 3600` over **all time**, descending. Ties $\rightarrow$ array order (stable sort $\rightarrow$ effectively insertion order from the merge). `rank = index + 1`.

10.2 How it's computed

`routes/leaderboard.js` :

1. `User.find({}).select('_id name email profilePictureUrl').lean()` — **all users**.
2. `Activity.aggregate` over the **entire collection**:
 - `$addField` `userIdObj` — `$regexMatch` for a 24-hex string $\rightarrow$ `$toObjectId`, else keep (works around `userId` being a `String`).
 - `$group` by `userIdObj`: `$sum duration`, `$sum linesAdded/Removed`, `$addToSet projectName`, `$sum 1` (count).
 - `$addField`: `totalHours`, `projectCount = $size(projects)`, `codeChanges`, `netCodeChanges`.
3. Build a `userId  $\rightarrow$  stats` map in Node; `map` all users onto it (users with no activity get zeros).
4. Sort by `totalHours` desc; assign `rank`.
5. **Relative scoring**: `maxHours`, `maxChanges`, `maxCommits` (where "commits" = activity-doc count). Each user's `speed/quality/engagement/impact = min(5, ratio*5)`; `overall` = their average; `commitScore = min(5, commits/20*5)`.

10.3 Properties & problems

- **Dynamic, never cached** — recomputed every request $\rightarrow$ always fresh, always expensive.
- **No time window** — cost grows with total activity **forever**.
- **No pagination, no \$limit** — the entire ranked list is returned.
- **Relative scores are unstable** — everyone's score shifts when the top user codes.
- `$addToSet` **on every project name** across all activity — memory-heavy in the pipeline.
- **Privacy**: every row includes `email`.
- **"commits" is a lie** — it's `activityCount`, not `gitAnalytics.commits`.
- **Group leaderboard** (`/groups/:id/details`) — same pattern, scoped to member IDs, all-time, no pagination; the `Promise.all(members.map(async ...))` has no `await` inside so the `async` is pointless.

10.4 Complexity

$O(A + U)$ per request where A = total activity documents ever written, U = total users; plus $O(U \log U)$ sort; all in application memory. On Render's request timeout / Vercel's serverless limit this fails "long before the user count becomes interesting" (H-7).

10.5 "Optimise for 10M users" [RECOMMENDED IMPROVEMENT]

1. **UserStats rollup collection** — `{ userId, totalSeconds, totalLinesAdded, projectCount, activityCount, updatedAt }`, incremented on ingest with `$inc` (or by a scheduled job). Leaderboard = `UserStats.find().sort({totalSeconds:-1}).limit(50)` — **O(index scan)**, reads `U` docs not `A`.
2. **Time-windowed** rollups (`daily_stats` keyed `{userId, day}`) for "this week/month" boards; sum the window.
3. **Cache** the top-N list in Redis with a short TTL; recompute on a cron or on write.
4. **Paginate / cursor** the full board.
5. **Rank at read time** from the sorted rollup, or maintain a Redis **sorted set** (`ZADD leaderboard <score> <userId>`, `ZREVRANGE` for a page, `ZREVRANK` for "your rank") — $O(\log N)$ updates, $O(\log N + \text{page})$ reads, the classic answer.
6. **Anti-cheat** before any of this matters (see §26).

11. Groups

11.1 Model [ACTUAL IMPLEMENTATION]

`groups` (metadata + script-hashed `password`, `select:false`) + `groupmembers` join table ({`groupId`,`userId`} unique compound). Membership is **not** an embedded array (unlike `teams`), which is the right call for a growing many-to-many.

11.2 Lifecycle

- **Create** (`POST /groups/create`): validate name/description/visibility; private → require a password, `hashPassword(password)` (script); `Group.save`; auto-add creator as a `GroupMember`; response strips `password`.
- **Discover** (`GET /groups/discover`): `Group.find({ _id: { $nin: joinedGroupIds } })` optionally `name: { $regex: search, $options:'i' }`.
- **Join** (`POST /groups/:id/join`): `Group.findById().select('+password')`; reject if already a member; private → `verifyPassword(password, group.password)` (401 on mismatch); **opportunistic migration** — if the stored value isn't a hash, rehash it now; `GroupMember.save`. Public group → `handleJoinClick` in the UI POSTs an empty body and the backend skips the password check.
- **Details** (`GET /groups/:id/details`): **membership check first** (`GroupMember.findOne` → 403 if not a member); then members list + a group leaderboard (all-time `Activity.aggregate` over member IDs).
- **Leave** (`POST /groups/:id/leave`): `GroupMember.findOneAndDelete`; if `countDocuments({groupId}) === 0` → `Group.findByIdAndDelete` (group self-destructs when empty).

11.3 Authorization & concurrency issues

- **Ownership**: only membership is checked. There's no "admin" concept for groups — `createdBy` is stored but never used to gate anything. Any member can view details; there's no kick/rename/delete-by-owner, no transfer.
- **Double-join race**: unique index catches it → **500** (should be 409).
- **Password brute force**: no rate limit on `/join` → a private group's password can be guessed offline-speed via the endpoint.
- **Empty-group deletion race**: two `leave`s at once → both compute `remainingMembers`, one deletes; harmless but not transactional.
- **Enumeration**: `discover` lists every non-joined group including private ones (name + description + creator), so private groups aren't hidden, only password-gated.

11.4 Interview questions this raises

"How do you prevent a user joining twice?" (unique index) · "What status code should a duplicate join return?" (409) · "How would you add group admins?" (a `role` column on `groupmembers`, gate mutations on `role === 'owner'`) · "Private group password vs per-user invites — which is better and why?" (invites: revocable, auditable, no shared secret) · "How would you rate-limit join attempts?" (per-user + per-group counter, exponential backoff).

12. Insights / ML system

12.1 The honest classification [ACTUAL IMPLEMENTATION]

It is deterministic descriptive statistics, not machine learning. No model, no training, no inference, no Python, no LLM, no external AI API. Everything is pure JavaScript in `services/metricsDerive.js` (unit-tested, dependency-free) fed by aggregations in `services/metricsService.js`.

12.2 The pipeline

```
GET /api/metrics?days=30&timezone=<offset>
→ isAuthenticated (identity = session only, no :userId)
→ buildMetrics(userId, {days, timezoneOffset})
  (A) Activity.aggregate $group over {userId, timestamp ≥ now-days}:
    focusedMs, flowBlocksMs (pushed+flattened), churnLines, linesInserted,
    readMs, writeMs, fileSwitches, commits, totalSeconds
  (B) $group by local day → daily minutes[]
  (C) $group by local hour → [{hour, minutes, commits, linesInserted, churnLines}]
  (D) buildGoalPairs: Goal.find({status:'completed'}) → Activity.aggregate by language
    → [{estimatedHours, actualHours}]
→ metricsDerive:
  deepWorkRatio      =  $\Sigma(\text{block} \geq 25\text{min}) / \text{focusedMs}$   $\in [0,1]$ 
  flowBlockStats     = { medianMs, longestMs, deepBlockCount, blockCount }
  consistencyIndex   = clamp( $1 - \text{stddev}(\text{dailyMinutes}) / \text{mean}(\text{dailyMinutes})$ , 0, 1)
  truePeakWindow     = argmax_hour(  $\text{commits} * 10 + \text{linesInserted} / 10 - \text{churnLines} / 5$  )
  estimationCalibration = mean(actualHours / estimatedHours) over completed goals (≥2)
    + churnRatio, comprehensionLoad, contextSwitchesPerHour, commits, totalHours
→ res.json({ success, metrics })
```

12.3 Every metric, defined exactly

Metric	Formula (as coded)	Interpretation	Failure mode
deepWorkRatio	$\Sigma \text{ blocks} \geq 25 \text{ min} \div \text{totalFocusedMs}$, rounded 2dp	fraction of focused time in long stretches	0 if no focus data or $\text{focusedMs} \leq 0$
flowBlocks	median / longest / deep-count / count of <code>flowBlocksMs</code>	shape of sessions	zeros on empty
consistencyIndex	$1 - (\sigma/\mu)$ of daily minutes, clamped $[0,1]$	steadiness of daily habit	0 if no days or $\mu \leq 0$
truePeakWindow	hour maximising $\text{commits} \cdot 10 + \text{linesInserted} / 10 - \text{churn} / 5$ among hours with $\text{minutes} > 0$	most <i>productive</i> hour (vs busiest)	null if no hours with tracked time
estimationCalibration	mean(actual/estimated) over completed goals with estimated > 0	>1 = you underestimate	null if < 2 usable pairs
churnRatio	$\text{churnLines} / \text{linesInserted}$	rework fraction	0 if $\text{linesInserted} = 0$
comprehensionLoad	$\text{readMs} / (\text{readMs} + \text{writeMs})$	share of time reading	0 if no attention data
contextSwitchesPerHour	$\text{fileSwitches} / \text{focusedHours}$	fragmentation	0 if $\text{focusedHours} = 0$

12.4 Sync/async, caching, regeneration

- **Synchronous** — the request awaits four aggregations then returns.
- **Not cached** — recomputed on every page load and every window change.
- **Not scheduled** — there's no background job producing insights.
- **Failure:** caught → `500 {success:false}` ; the page shows an error card + "Try again".
- **Insufficient data:** `flowBlocks.blockCount === 0` → focus/flow/churn/read/switch cards render "—" plus a "needs extension 2.1.0" banner; `estimationCalibration null` → prompt to complete 2 goals.

12.5 "Is this really machine learning?" — the answer

"No, and I wouldn't claim it is. It's descriptive statistics computed deterministically: coefficient of variation for the consistency score, a weighted linear score for the productive-hour ranking, medians for flow blocks, a ratio for estimation calibration. There's no model, no training data, no inference. I chose that deliberately — every number on the page is explainable and reproducible, and it works from day one with no cold-start problem. The design doc has an LLM layer on top that would narrate these metrics — 'your afternoons are busy but churning, your evenings are where you actually ship' — with a hard rule that any number it states must be traceable to the metrics object, to stop it hallucinating. That's specced (Gemini was the pick) but not built."

12.6 If asked to "make it ML" [RECOMMENDED IMPROVEMENT]

- **Feature engineering:** per-day/session vectors — focused minutes, deep-block count, churn ratio, commits, context switches, hour-of-day, day-of-week, language mix.
- **Preprocessing:** per-user z-score normalisation (habits are relative), handle missing focus data (pre-2.1.0), aggregate to session grain.
- **Unsupervised first:** k-means / GMM over session vectors → "session archetypes" (deep-focus, exploratory, firefighting); PCA for a 2-D "session map".
- **Supervised only if you have a label:** predict next-day minutes (regression) or goal-slip risk (classification) — needs labelled outcomes you don't currently store.
- **Anomaly detection** on ingest for anti-cheat (isolation forest on `duration` vs `edits`).
- **Serving:** a separate Python/FastAPI service or batch job writing an `insights` collection; never inline in the request.
- **Evaluation:** hold-out RMSE / AUC; guard against leakage (don't train on the same window you predict); monitor drift.
- **The LLM layer:** structured-metrics-only prompt, JSON-schema-constrained output, a validator that rejects any figure not present in the metrics object, a cache keyed on `(userId, window, metricsHash)`.

13. Authentication & authorization

13.1 Web auth flow [ACTUAL IMPLEMENTATION]

1. `Login.tsx` → `window.location = API_URL + '/auth/google'`.
2. `GET /auth/google` → `passport.authenticate('google', { scope:['profile','email'], session:false })`.
3. Google redirects to `/auth/google/callback`. Passport's verify callback find-or-creates the `User` (new → `generateApiKey()`), returns the user.
4. Handler signs `jwt.sign({id,name,email,isFirstLogin}, JWT_SECRET || 'your_jwt_secret', {expiresIn:'1d'})`, sets cookie `token` (`httpOnly`; `prod: secure + sameSite:'none'`; `dev: lax`; `maxAge 24 h`), redirects to `FRONTEND_URL + '/onboarding' | '/dashboard'`.
5. Frontend `App.tsx` → `GET /api/user/profile (credentials:'include')` — 200 renders the app, anything else → `/login`.
6. `isAuthenticated` on protected routes: `jwt.verify(req.cookies.token)` → `User.findById(decoded.id)` → `req.user`.
7. `POST /auth/logout` clears the cookie.

13.2 Extension auth

API key in `x-api-key`; `verifyApiKey` → `User.findOne({ apiKey })`. See §5.

13.3 Authorization [ACTUAL IMPLEMENTATION]

- `services/authorization.js`:
 - `sameUser(a,b)` — `String(a) === String(b)`, null-safe.
 - `assertOwnership(requestedId, sessionId)` → `{ok}` if IDs match or `requestedId` is falsy; `403` if they differ; `401` if no session.
 - `isBypassAllowed(env)` — `AUTH_BYPASS === 'true' && NODE_ENV !== 'production'`.
- **Analytics routes:** `resolveOwnedUserId(req,res)` calls `assertOwnership` → `403` on mismatch (H-1 fix).
- **Metrics:** no `:userId` at all — identity is the session (`req.user._id`).
- **Goals:** `Goal.findOne({ _id: goalId, userId: req.user.id })` — scoped (H-11 fix).
- **Teams:** `GET /:teamId` checks `team.members.some(sameUser(..., req.user.id))` → `403`; `add-member` checks `team.createdBy === req.user.id` → `403`.
- **Groups:** `:groupId/details` checks `GroupMember.findOne` → `403`.
- **Notifications:** every query scoped by `req.user._id`.

13.4 Gaps

- No refresh token — 24 h then a hard logout.
- No server-side revocation / session list — a stolen JWT is valid until expiry.
- `JWT_SECRET` fallback (M-9).

- `sameSite: 'none'` in production + no CSRF token — limited exposure because most mutations are `POST` with a JSON body and `credentials: 'include'`, but `POST /auth/logout` and the no-body `POST`s are technically CSRF-able (low impact).
- Group `createdBy` is stored but never used for authz.

14. Security analysis

Format: **Current** → **Vulnerability** → **Attack** → **Fix**.

14.1 API key — plaintext bearer credential

- **Current:** 256-bit random, stored plaintext, compared with `findOne({apiKey})`, no expiry, no scope, sent every request, shown in the dashboard and pasted into `settings.json`.
- **Vulnerability:** DB read → all keys usable; key in logs/history/dotfiles → account ingest hijack.
- **Attack:** `for i in {1..1000}; do curl -XPOST ../api/extension/track -H "x-api-key: $K" -d '{"fileName":"a.ts","language":"TS","duration":3600}'; done` → victim tops the leaderboard; their analytics are garbage.
- **Fix:** hash at rest (`keyId` +secret, prefix), per-device keys, rotation with grace, scope, `lastUsedAt`; or short-lived signed ingest tokens; rate-limit + validate + idempotency on ingest.

14.2 No rate limiting anywhere

- **Current:** `express-rate-limit` installed, never used. `/auth/google`, `/api/extension/track`, `/api/groups/:id/join` all uncapped.
- **Attack:** credential-stuffing the OAuth callback; ingest flooding; private-group password brute force.
- **Fix:** `express-rate-limit` — global default + tighter on `/auth` and `/api/extension` and `/join`; per-key limits on ingest.

14.3 No security headers

- **Current:** `helmet` installed, never used. No HSTS, no `X-Content-Type-Options`, no `X-Frame-Options`, no CSP.
- **Fix:** `app.use(helmet())` (the old `server.js.old` did this — it was dropped in the rewrite).

14.4 No input validation on ingest

- **Current:** only `if (!fileName || !language || !duration); Number(duration)` accepts `1e12`, negatives (partly clamped by normalisers for sub-docs, not for `duration` itself), and a client-supplied `timestamp`.
- **Attack:** one request writes `duration: 999999999` → instant leaderboard #1; backdate `timestamp` to fabricate history.
- **Fix:** `express-validator` — `duration` `int 1..3600`, `timestamp` within `[now-24h, now]`, `language` / `fileName` length caps, reject unknown top-level keys.

14.5 Error messages leak internals

- **Current:** `res.status(500).json({ message, error: error.message })` in every route.
- **Attack:** provoke errors to learn stack shapes, field names, Mongo error text.
- **Fix:** central error middleware — log server-side with a request id, return `{ error: 'Internal error', id }`.

14.6 Leaderboard exposes every user's email

- **Current:** `GET /api/leaderboard` returns `email` per row.
- **Fix:** return a display name / handle only.

14.7 AUTH_BYPASS — fixed, but note it

- **Current:** `isBypassAllowed` returns false when `NODE_ENV==='production'`, and the middleware refuses it; tests assert this (H-10).
- **Residual:** still a big footgun in dev; anyone running the API with `NODE_ENV` unset and `AUTH_BYPASS=true` is fully open.

14.8 JWT

- **Current:** `HS256`, `JWT_SECRET || 'your_jwt_secret'`, 1-day, `httpOnly` cookie, no refresh, no revocation.
- **Attack:** if the fallback secret is ever live, anyone can forge a token for any `id`.
- **Fix:** fail fast if `JWT_SECRET` is unset (M-9); add refresh + a rotating secret or `RS256`.

14.9 No idempotency / replay protection on ingest

- **Attack:** capture one valid POST, replay it N times → N× the activity.
- **Fix:** client `idempotencyKey` (uuid per flush) + a unique index / short-lived dedupe set.

14.10 MongoDB injection

- **Current:** `Group.find({ name: { $regex: search, $options: 'i' } })` — `search` is user input used as a regex → **ReDoS** risk (a pathological pattern) and it's an unescaped regex, not a literal.
- **Fix:** escape the input or use `$text` / an anchored literal; cap length.
- Elsewhere queries pass values as query *values* (not `$where`, not string-concatenated), so classic NoSQL operator injection is limited — but `express.json()` will happily parse `{"apiKey": {"$ne": null}}` as a body; `verifyApiKey` does `typeof apiKeyHeader === 'string'` on the **header**, so that path is safe. Worth auditing any place a raw body object reaches a query.

14.11 XSS / CSRF

- **XSS:** React escapes by default; no `dangerouslySetInnerHTML` in the codebase → low risk. User-controlled strings (group names, goal titles) render as text.
- **CSRF:** `sameSite: 'none'` + no token → the no-body `POST`s are theoretically forgeable; impact is low (logout, onboarding flag, key regeneration — the last is a mild annoyance vector). Add a CSRF token or `sameSite: 'lax'` + a separate ingest origin.

14.12 Extension security

- Key in `settings.json` not `SecretStorage`.
- Command strings sanitised before transmit (`token=***`), truncated to 120 chars — good.

- Never transmits file contents, diffs, commit messages, or absolute paths — verified by `activation.test.js` (payload must not contain the workspace path).

15. Error handling & reliability

Failure	What happens now [ACTUAL]	Weakness	Fix [RECOMMENDED]
MongoDB down at boot	<code>mongoose.connect().catch(console.error)</code> — process keeps running	every request then throws a generic 500	health check that fails readiness; retry with backoff; <code>/health</code> reports DB state (the old server did)
MongoDB down mid-request	<code>route try/catch → 500 { error: err.message }</code>	leaks Mongo text; no retry	central handler + transient-error retry
Ingest: malformed payload	normalisers coerce sub-docs to 0; missing <code>fileName/language/duration → 400</code>	<code>duration:0</code> rejected; <code>1e12</code> accepted; junk <code>timestamp → now</code>	<code>express-validator</code> with bounds
Ingest: invalid key	<code>401 { message: 'Invalid API key' }</code>	fine	+ rate-limit to slow enumeration
Extension offline	buffered minutes kept in memory, retried next tick; best-effort flush on deactivate	restart loses un-flushed time	persist the buffer to <code>context.globalState</code> ; a small on-disk queue
Frontend API failure	<code>console.error</code> , page stays in empty/loading or shows an error card	inconsistent; <code>Groups</code> uses <code>alert()</code>	a shared fetch hook with typed errors + toasts; React Query retry
Metrics/ML failure	<code>500 {success:false} → "Could not load your insights" + retry</code>	fine	+ a cached last-good result
Notification cron on serverless	never fires (frozen between requests); cold start re-runs the immediate sweep	duplicate reminders / none at all (H-13)	Vercel Cron / external scheduler hitting a protected internal route
Unhandled rejection	Express 5 turns it into a 500	no logging context	central handler + <code>process.on('unhandledRejection')</code>

16. Performance & scalability (10k → 1M users)

Assume ~2 active hours/user/day → ~ 120 flushes/user/day → ~ 1 write/user/min while active.

10,000 users [ACTUAL: mostly fine]

- **Writes:** ~ a few hundred inserts/min peak — trivial for one Mongo node.
- **Leaderboard:** `activities` might be ~ 10^7 – 10^8 docs after months. The all-collection `$group` + `$addToSet` + load-all-users starts taking seconds and risks the request timeout. **First thing to break (H-7).**
- **Analytics:** `find(7d)` per user is bounded (~ 1k docs) but every call ships and parses them all (M-1). Add `{userId:1,timestamp:-1}`.
- **Frontend:** every nav refetches — noticeable but survivable.

100,000 users

- **Leaderboard is dead** without a rollup.
- **Ingest** ~ thousands of writes/min — one Mongo primary is still okay but the connection pool on a small Render instance is a bottleneck; the synchronous `Activity.create` per request ties up an event-loop turn on JSON parse + write ack.
- **No caching** → repeated identical analytics/metrics recomputation.
- `node-cron` sweeping all overdue goals with an unindexed `findOne` per goal is now expensive.

1,000,000 users

- **Single monolith + single Mongo is the wrong shape.** Need:

```
Extensions → API (stateless, autoscaled) → Queue (SQS/Kafka) → Ingest workers → activities (sharded by
userId)
                                                    ↳ $inc UserStats / daily_stats

rollups
Dashboard → API → read from rollups + Redis cache
Leaderboard → Redis sorted set (ZADD on rollup update)
Insights → batch worker writes `insights` collection nightly; API serves cached
```

- **Shard** `activities` **on** `userId` (hashed) — all per-user queries are single-shard.
- **Rollups** (`UserStats` , `daily_stats`) so reads are $O(\text{users})$ not $O(\text{activity})$.
- **Redis:** leaderboard sorted set; per-(user>window) insights cache; rate-limit counters.
- **Queue** decouples ingest spikes from write latency and lets you replay/repair.
- **CDN + React Query** on the frontend; HTTP caching headers on analytics.
- **Archival:** raw `activities` older than N months → cold storage / a columnar warehouse (BigQuery/ClickHouse) for trend analysis.
- **Observability:** structured logs, RED metrics, tracing — currently zero.

What to say

"The architecture is fine to about 10k users. The first thing that breaks is the leaderboard, because it aggregates the entire activity collection in application memory on every request. The fix is a `UserStats` rollup updated on ingest, which turns an $O(\text{all activity})$ scan into an $O(\text{users})$ indexed read, and a Redis sorted set if I want $O(\log n)$ rank lookups. Past ~100k I'd put a queue in front of ingest so write spikes don't raise latency, shard activities by `userId`, and move analytics aggregation from Node into MongoDB pipelines or precomputed rollups. The read path would sit behind Redis and React Query."

17. Testing

17.1 What exists [ACTUAL IMPLEMENTATION]

Framework: none. Plain `node:assert` scripts with a hand-rolled `check(name, fn)` counter, run via `node tests/x.test.js . package.json test` scripts chain them with `&&`.

Backend (backend/tests/ , ~57 assertions):

Suite	Technique	Covers
<code>streak.test.js</code>	extracts <code>localDayInfo</code> + <code>computeStreak</code> from the <i>shipped</i> <code>analytics.js</code> via string slicing, runs them against a stubbed <code>Activity.aggregate</code>	streak correctness incl. one regression case per historical bug; timezone day-bucketing
<code>ingest.test.js</code>	unit	<code>activityNormalizers</code> — defaults, number coercion, negative rejection, <code>flowBlocksMs</code> cap at 200
<code>metrics.test.js</code>	unit	all five <code>metricsDerive</code> functions incl. edge cases (empty, div-by-zero, even-length median, "productive vs busy" hour)
<code>authorization.test.js</code>	unit	<code>sameUser</code> , <code>assertOwnership</code> (403 vs 404 vs 401), <code>isBypassAllowed</code> (never in prod)
<code>routeGuard.test.js</code>	static source scan — regex over route files	fails if any sensitive route loses <code>isAuthenticated</code> / ownership; the H-1 regression guard
<code>passwordHash.test.js</code>	unit (async)	script round-trip, wrong password, salting, legacy plaintext acceptance, null-safety

Extension (extension/tests/):

Suite	Technique	Covers
<code>tracker.test.js</code>	unit against the built dist/extension.js + a <code>vscodeStub</code>	EditorTracker (gross counts, replace-in-place regression, churn window, undo/redo, attention split, large-insert-no-text), FocusTracker (focus/blur accrual, block close/gap/open), GitStateTracker (commit on HEAD move, no double-count, uncommitted age, no-op without git)
<code>activation.test.js</code>	loads the real bundle vs stub, activates it	every contributed command registered; no undeclared commands; no network call without a key ; payload has the 4 analytics blocks + gross line fields; no absolute path leak; manifest sanity (apiBase not localhost, prepublish compiles)

Frontend: zero tests.

17.2 Gaps

- No integration tests (nothing boots Express + hits a real/in-memory Mongo).
- No API contract tests (supertest).
- No end-to-end (extension → API → DB → dashboard).
- No frontend component/hook tests.
- Nothing has run against a live database — all backend verification is unit-level, pipelines are "syntax-checked and logically verified only" (session log).

17.3 Recommended strategy [RECOMMENDED IMPROVEMENT]

- **Unit (keep the style, add a runner):** move to `node:test` or Vitest; cover `buildTerminalSummary`, `localDayInfo`, `resolveOwnedUserId`, `computeStreak` against `mongodb-memory-server`.
- **Integration (backend):** `supertest` + `mongodb-memory-server`. Cases:
 - ingest with a valid key writes exactly one doc with normalised sub-docs;
 - ingest with a bad key → 401; missing fields → 400;
 - `GET /api/analytics/:otherUserId` with my cookie → 403;
 - `GET /api/metrics` returns zeros for a fresh user and non-zero after seeded activity;
 - leaderboard ordering with 3 seeded users;
 - group: create → creator is a member → second user joins with the right/wrong password → third user `/details` → 403 (not a member);
 - notification: mark-as-read only affects my rows.
- **Frontend:** React Testing Library — `App` renders `Login` when profile 401s; `Insights` shows the "2.1.0" banner when `blockCount === 0`; `Dashboard` toggles daily/weekly; `NotificationPanel` badge count.
- **Extension:** extend the stub to fire full event sequences; assert flush cadence, idle pause/resume, retry-keeps-buffer, deactivate final flush.
- **ML/metrics:** property tests — `consistencyIndex ∈ [0,1]`, `deepWorkRatio ≤ 1`, `truePeakWindow.hour ∈ 0..23`; golden-file test of `buildMetrics` against a fixed seeded DB.
- **E2E:** Playwright against a compose stack (Mongo + API + built frontend), plus a headless extension host test (`@vscode/test-electron`, already a devDep).
- **CI:** GitHub Action — `tsc -b` (must pass, so fix M-13 first), both test suites, `node --check` on backend, `vsce package --no-dependencies` dry run.

18. Debugging playbook

18.1 "A user's VS Code activity isn't on the dashboard."

Walk it in order (this is `CodeTrackr_Architecture.md` §9 as a checklist):

1. **Extension running?** `CodeTrackr: Show Connection Info` → backend URL, masked key, live status. `Show Statistics` → pending minutes, counters. If "Paused (idle)" — expected.
2. **API key valid?** `showInfo` calls `GET /api/extension/verify`. 401 → regenerate on Profile, re-run Setup API Key. Check for a trailing space (the code `.trim()` s the header but not always the stored value historically).
3. **Right backend?** `codetrackr.apiBase` — the 2.0.x regression pointed it at localhost. Should be `https://codetrackr-backend-uckp.onrender.com` (or your deploy).
4. **Is it flushing?** Dev tools / `console` in the Extension Host: look for "Activity tracked [fixed]" or "flush failed". `duration < minFlushMinutes` (0.5) → nothing sent yet. Run `CodeTrackr: Flush Now`.
5. **Network:** backend reachable? Render free tier asleep (first request ~30 s)? `axios` 15 s timeout — a cold start can exceed it once.
6. **Backend logs:** "Track activity error" → 400 (missing field) or 500 (Mongo). Confirm the doc landed: query `activities` by `userId` (the **string** hex) + recent `timestamp`.
7. **Dashboard fetch:** DevTools Network → `GET /api/analytics/<id>` — 401 (session expired → re-login), 403 (wrong `<id>` — should be `user.id` from the profile), missing `credentials:'include'` (was the H-1 bug).
8. **Timezone / "wrong day":** a flush near local midnight can bucket to the previous/next day; the daily view is *today in the browser's local day*. Check `?timezone=` is sent (it's `new Date().getTimezoneOffset()`).
9. **It's actually there but hidden:** "Repeated Failures" is mock data; Insights focus cards show "—" until 2.1.0 data exists; weekly vs daily toggle.

18.2 More scenarios (give a step-by-step for each in an interview)

- **"Leaderboard is slow / times out."** → it aggregates the entire `activities` collection per request (H-7); confirm collection size (`db.activities.countDocuments()`); short-term add a 30-day `$match` + `$limit` in the pipeline; medium-term a `UserStats` rollup.
- **"Two users have the same rank."** → ties break on array order; `rank = index + 1` with no tie handling; decide the tie-break rule (e.g. more recent activity, then `userId`).
- **"My streak reset even though I coded yesterday."** → `computeStreak` anchors on today *or yesterday*; check the 90-day window, the timezone offset, and whether yesterday's flush had `duration > 0` (the `$match: { seconds: { $gt: 0 } }`).
- **"Insights says 0% deep work."** → `flowBlocks.blockCount` — if 0, they're on a pre-2.1.0 extension; the UI should show "—" not "0%" (it does, gated on `hasFocusData`).
- **"Goal progress looks wrong."** → progress only counts activity where `language === goal.techStack` **exactly** (case-sensitive string match) and `timestamp <= deadline`; a goal with `techStack: 'React'` won't match `language: 'javascript'`.

- **"Notifications never arrive."** → is the backend serverless? `node-cron` won't fire (H-13). On Render, check the process stayed up (free tier sleeps).
- **"`npm run build` fails on the frontend."** → `tsc -b` 29 errors (M-13); `vite build` alone would succeed.
- **"Duplicate activity after a flaky network."** → no idempotency; the retry re-sent and the backend stored both.

19. Deployment / DevOps

19.1 What's in the repo [ACTUAL IMPLEMENTATION]

- **Frontend:** `frontend/vercel.json` — SPA rewrite `/(.*) → /index.html`. Build: `tsc -b && vite build` (**currently red**). Env: `VITE_API_URL`.
- **Backend:** `backend/vercel.json` — `@vercel/node` build of `api/index.js` (which wraps `app.js` with `serverless-http`), route-all to it. Also runnable as a plain process (`node app.js`, `app.listen(PORT || 5050)`). The extension's default `apiBase` points at **Render** (`codetrackr-backend-uckp.onrender.com`) — so the *live* backend is most likely Render, with the Vercel config as an alternative/earlier setup.
- **DB:** MongoDB Atlas (`MONGO_URI`, TLS, `family:4`, 15 s selection timeout).
- **Extension:** `vsce package` → `.vsix`; publisher `CodeTrackr-ext`; `codetrackr-vscode-2.2.0.vsix` committed. `PUBLISHING_v2.0.0.md` documents the Azure DevOps PAT + `vsce publish` flow.
- **No** Dockerfile, Procfile, `render.yaml`, GitHub Actions, or any CI.

19.2 Env vars

`backend/.env.example`: `MONGO_URI`, `JWT_SECRET` (unsafe fallback), `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_CALLBACK_URL`, `FRONTEND_URL`, `PORT`, `NODE_ENV`, `AUTH_BYPASS` (dev only). `config/passport.js` **throws at import** if the three `GOOGLE_*` are missing — the API won't boot without them.

19.3 The serverless / scheduler mismatch (H-13)

`app.js` calls `initScheduler()` and `app.listen()` unconditionally. On Vercel serverless: every cold start re-runs the immediate `checkUpcomingDeadlines()` + `checkOverdueGoals()` sweep; the hourly `cron.schedule('0 * * * *')` never fires because the process is frozen between requests. Fix: guard `app.listen()` behind `require.main === module`; move the schedule to Vercel Cron / an external trigger hitting a protected `POST /api/internal/run-notifications`; add `{ goalId:1, type:1 }` index for `checkOverdueGoals`'s per-goal `findOne`.

19.4 How I'd deploy it properly [RECOMMENDED IMPROVEMENT]

- Backend: a single Docker image on Render/Fly/Railway (long-lived process → `node-cron` works), or Vercel serverless + Vercel Cron for the schedule. Health check on `/`.
- Frontend: Vercel/Netlify, `VITE_API_URL` per environment, preview deploys per PR.
- DB: Atlas M10+ with backups; a separate cluster per environment.
- Secrets: platform secret stores, never `.env` in git (it's `.gitignore`d here — verified).
- CI: GitHub Actions — lint + `tsc -b` + both test suites on PR; on tag, build the `.vsix` and `vsce publish` with a PAT in Actions secrets; deploy hooks for Render + Vercel.
- Observability: a hosted logger (structured JSON), uptime checks, error tracking (Sentry).

20. Design decisions — "why did I choose this?"

Each: **Decision** → **Reason** → **Alternative** → **Trade-off** → **When the alternative wins**.

D1. MongoDB (not Postgres)

- **Reason:** append-only, self-contained, schema-evolving activity docs; per-user-window access; no hot-path joins; free managed hosting; team familiarity.
- **Alternative:** Postgres (+ Timescale for the stream).
- **Trade-off:** no transactions; `userId` type mismatch; the leaderboard scan; weaker relational integrity for groups/teams/goals.
- **Postgres wins when:** the relational features grow (roles, invites, cascades), the leaderboard needs `GROUP BY` at scale, or you want real analytics SQL.

D2. Modular monolith (not microservices)

- **Reason:** one dev, one deploy, one DB, shared model; features aren't independently scalable yet.
- **Alternative:** ingest service + analytics service + web BFF.
- **Trade-off:** the whole thing scales together; a slow leaderboard query can starve ingest.
- **Microservices win when:** ingest volume dwarfs everything (extract it + a queue), or teams need independent deploys.

D3. API key for the extension (not OAuth / JWT)

- **Reason:** no login UI in VS Code; one `findOne`; works offline once configured; trivial to regenerate.
- **Alternative:** OAuth device flow; short-lived signed ingest tokens; a JWT the dashboard mints.
- **Trade-off:** unscoped, non-expiring, plaintext bearer credential; leak = ingest hijack; all-or-nothing rotation.
- **The alternative wins:** basically always for production — but the key model is a legitimate MVP. (Full redesign in §5.4.)

D4. Google OAuth → JWT in an httpOnly cookie (web)

- **Reason:** no password storage; stateless; survives serverless cold starts; `httpOnly` blocks JS theft.
- **Alternative:** server-side sessions; a managed auth provider.
- **Trade-off:** no revocation; no refresh; `JWT_SECRET` fallback; CSRF surface with `sameSite: 'none'`.
- **Sessions win when:** you need instant logout-everywhere or per-session device management.

D5. One `Activity` document per flush (not upsert/merge per day)

- **Reason:** simplest possible write; immutable; no read-modify-write races; keeps per-flush granularity (10-minute drill-down needs it).

- **Alternative:** daily/hourly bucket docs with `$inc` (the `server.js.old` approach); a time-series collection.
- **Trade-off:** document count explodes; reads must aggregate many small docs.
- **Buckets win when:** storage/read cost matters — i.e. at scale, as a rollup *alongside* the raw stream.

D6. Aggregate analytics in Node (not MongoDB `$group`)

- **Reason:** honestly, expedience — `find().then(reduce)` was faster to write and easier to debug while the metric definitions were changing.
- **Alternative:** `$match / $group / $facet` pipelines (the metrics service already does this).
- **Trade-off:** ships and parses every document; memory $O(N)$; the M-1 tech-debt item.
- **`$group` wins:** always, once the definitions stabilise — it's the prerequisite for scale.

D7. Deterministic statistics for Insights (not ML / not an LLM)

- **Reason:** explainable, reproducible, no training data, no cold start, cheap; every number defensible in a review.
- **Alternative:** clustering for session archetypes; an LLM narration layer (designed).
- **Trade-off:** no personalised recommendations, no natural-language summary, no learning.
- **ML wins when:** you have labelled outcomes (goal slippage, retention) and enough users; the LLM layer wins for UX once there's a validator to stop hallucinated numbers.

D8. `groupmembers` join table but `teams.members` embedded array

- **Reason:** groups are the real feature (discover, join, leaderboard) and need efficient membership queries + a uniqueness guard; teams were an earlier, smaller idea.
- **Trade-off:** two patterns for "membership"; `team.members.push` has a lost-update risk; inconsistent mental model.
- **Consolidate:** move teams to a join table too, or drop teams (the UI already doesn't use it).

D9. React + Vite (not Next.js)

- **Reason:** auth-gated app, no SEO need, no SSR need; Vite is the simplest fast build.
- **Alternative:** Next.js (routing, API routes, SSR), Remix.
- **Trade-off:** you assemble routing/data-fetching yourself (and here, didn't finish — no React Query despite it being installed).
- **Next wins when:** you want SSR/ISR, co-located API routes, or file-based routing.

D10. 28 runtime themes via a Context + inline styles

- **Reason:** a fun differentiator; DaisyUI-style palettes without DaisyUI.
- **Trade-off:** every component carries `style={{ color: theme.colors.x }}`; verbose; `tailwind.config` `extend` is empty so utilities can't use the theme.

- **Better:** CSS custom properties + Tailwind `theme.extend` referencing them → `text-primary` just works.

21. Known limitations

21.1 Current limitations (this is a class/side project)

#	Problem	Why it matters	Impact	Fix
1	API key plaintext, non-expiring, unscoped	it's a credential, not an id	key leak → ingest hijack, leaderboard fraud	hash at rest, scope, rotate, per-device
2	Leaderboard scans all activity + all users in Node	O(A+U) per request, no cache/window/pagination	times out at ~10k users	UserStats rollup + Redis sorted set
3	Analytics aggregate in JS	ships/parses every doc, O(N) memory	slow reads, wasted DB egress	MongoDB \$group pipelines
4	Missing {userId:1,timestamp:-1} index	every query filters timestamp but indexes are on date	full per-user scans	add the index; drop the date index
5	node-cron on serverless	frozen between requests	notifications never fire (H-13)	Vercel Cron / external trigger
6	No helmet / rate limit / validation	Express gives nothing free	header attacks, flooding, fake data	wire in the 3 installed deps
7	Frontend doesn't typecheck	npm run build fails at tsc -b	can't ship a clean build	fix the 29 errors (M-13)
8	Dashboard "Repeated Failures" is mock	shows fabricated commands	misleading UI	render terminalSummary.repeatedFailedCommands
9	Goals to-dos not persisted; no complete/delete	data vanishes on refresh	feature is half-built	add routes + wire the UI; call /progress
10	Teams UI orphaned	dead code, backend routes unused	confusion	route it or delete it
11	Idle < 2 min counts as active	totals skew high	inflated hours	count only intervals with a real edit event (L-7)
12	activities.userId String vs ObjectId elsewhere	coercion gymnastics, no \$lookup	leaderboard hack, brittle	migrate with a compat window (M-6)
13	Extension has no offline queue	un-flushed time lost on restart	data loss	persist buffer to globalState
14	No idempotency on ingest	retry = double count	inflated data	client idempotency key + unique index
15	Insights recomputed every request	4 aggregations per page load	slow page, DB load	cache per (user, window)
16	Errors echo err.message	internals leak	recon aid	central error middleware
17	Leaderboard exposes emails	PII in a shared list	privacy	return handles only
18	No integration/e2e/frontend tests; nothing run vs a real DB	pipelines unverified end-to-end	regressions slip	supertest + mongodb-memory-server, Playwright

21.2 Production-level concerns (would block a real launch)

- No observability (logs are console.log, no metrics, no tracing, no error tracking).
- No CI/CD; manual deploys; manual vsce publish (and it stalled on an expired PAT).

- No backups/DR story documented.
- No abuse/anti-cheat (leaderboard is trivially gamed — see §26).
- No data-retention / privacy policy for a tool that records developer behaviour; GDPR-style "export/delete my data" not implemented.
- Single points of failure: one Mongo, one API instance, Render free-tier sleep.
- Extension version-management incident (a 2.1.0 uploaded before 2.0.x meant the Marketplace served stale code for months) — needs a release checklist + CI publish.

22. Future improvements roadmap

Short term (days)

- Fix the 29 TS errors → green `npm run build`.
- `app.use(helmet())`; `express-rate-limit` on `/auth`, `/api/extension`, `/join`.
- `express-validator` bounds on `/track` (`0 < duration ≤ 3600`, `timestamp` sanity).
- Central error middleware; stop echoing `err.message`.
- Add `{userId:1,timestamp:-1}` index; backfill `date = timestamp`.
- Fail fast if `JWT_SECRET` unset.
- Wire real `repeatedFailedCommands` into the dashboard; delete or route `Teams.tsx`.
- Guard `app.listen()` behind `require.main === module`; move the cron to an external trigger.

Medium term (weeks)

- **API keys:** hash at rest, `keyId` prefix, per-device keys, rotation grace window, `lastUsedAt`.
- **UserStats rollup** updated on ingest (`$inc`) → leaderboard reads $O(\text{users})$; `?period=` + pagination.
- **Shared aggregation service** — one module doing `$group` for analytics *and* metrics; delete the JS reduce paths (M-1).
- **React Query** (already installed) for caching + dedup + retry across the frontend.
- **Extension offline queue** persisted to `globalState`; **idempotency key** on ingest.
- **Migrate** `activities.userId` → **ObjectId** with a dual-read compat window; then `$lookup` the leaderboard.
- **CI:** typecheck + both test suites + `.vsix` dry run on PR; auto-publish on tag.
- **Integration tests** with `supertest` + `mongodb-memory-server`.

Long term (months)

- Ingest → **queue** → **workers** → **sharded activities** + rollups; **Redis** for the leaderboard sorted set and the insights cache.
- **Analytics warehouse** (ClickHouse/BigQuery) fed from the stream for trends/cohorts; TTL/archival on raw docs.
- **The LLM insights layer** — structured-metrics-only prompt, JSON-schema output, a numbers-must-be-grounded validator, per-(user>window) cache.
- **Session-archetype clustering** and **anomaly detection** (anti-cheat) on ingest.
- **Personalised recommendations** ("your 3rd hour is your worst — schedule reviews then").
- **Observability** everywhere; **SLOs**; **data export/delete** for privacy compliance.
- **Per-device / team dashboards**, real-time group activity over WebSockets.

23. Time & space complexity

Let N = a user's activity docs in the query window; A = all activity docs ever; U = all users; D = days in window; B = flow blocks; G = a user's goals.

Operation	Time	Space	Notes / can it be better?
POST /track ingest	$O(1)$ (one insert)	$O(1)$	fine
POST /track/batch	$O(k)$ inserts	$O(k)$	insertMany ; unused
verifyApiKey	$O(\log U)$ index lookup	$O(1)$	fine (would be same on a hashed keyId)
isAuthenticated	$O(1)$ verify + $O(\log U)$ findById	$O(1)$	could cache the user for the request
Daily analytics	$O(N)$ transfer+parse + $O(N)$ reduce	$O(N)$ (all docs resident)	→ $O(N)$ in Mongo with \$group , $O(1)$ app memory
computeStreak	$O(N_{90})$ \$group + $O(90)$ walk	$O(\text{distinct days})$	good enough; index on timestamp would help the \$match
Weekly / timeslot	$O(N)$	$O(N)$	same as daily
/summary	$O(N)$ in Mongo (\$group)	$O(\text{days} + \text{langs})$	already a pipeline — the model to follow
Global leaderboard	$O(A)$ aggregate + $O(U)$ merge + $O(U \log U)$ sort	$O(A_{\text{projects}} + U)$	→ $O(U)$ read + $O(U \log U)$ sort from a rollup; $O(\log U)$ with a Redis ZSET
Group leaderboard	$O(A_{\text{member}})$ aggregate + $O(M)$	$O(M)$	same rollup idea, scoped
/api/metrics	$4 \times O(N)$ \$group + $O(B \log B)$ sort in flowBlockStats + $O(D)$ stddev + $O(24)$ peak + $O(G)$ pairs	$O(B + D)$	not cached — cache per (user, window)
consistencyIndex	$O(D)$	$O(D)$	
truePeakWindow	$O(24)$	$O(1)$	
estimationCalibration	$O(G)$	$O(G)$	
EditorTracker consumeChurn	$O(R)$ over recent inserts (bounded by 10-min window)	$O(R)$	fine
flowBlockStats median	$O(B \log B)$ sort	$O(B)$	B capped at 200 by the normaliser
Dashboard render	$O(N)$ dataset build per chart	$O(N)$	cheap; not memoised deliberately
Frontend nav	refetch every time, no cache	—	React Query would dedupe

Headline: the two costs that matter are (1) **leaderboard = $O(\text{all activity})$** and (2) **analytics = $O(N)$ app memory + DB egress**. Both have the same fix family: aggregate in the engine / precompute rollups.

24. Code-level questions (with answers)

middleware/auth.js — verifyApiKey

- *What does it do?* Extracts `x-api-key`, `User.findOne({ apiKey })`, attaches `req.user` or 401s. In `AUTH_BYPASS` (dev, non-prod) it attaches the most-active user instead.
- *Why `findOne` on a plaintext field?* Simplicity + the unique index makes it $O(\log n)$. It's the design weakness (§5).
- *Race condition?* No — read-only, idempotent.
- *Optimise?* Hash → `keyId` lookup + constant-time compare; cache `apiKey → userId` in Redis.
- *If it fails?* `catch → 500 { message: 'Failed to authenticate API key' }`.

routes/leaderboard.js — the aggregate

- *Why the `$regexMatch / $toObjectId`?* `activities.userId` is a `String`; some legacy rows may not be 24-hex; the pipeline coerces valid ones so the `$group` key matches `users._id`.
- *Time complexity?* $O(A)$ — every activity doc. *Space?* `$addToSet` on every project name.
- *Race?* No, but the result is different on every call (relative scoring).
- *Edge case:* zero users → `maxHours = 1` guard; `Math.max(...[], 1)` guards empty arrays.
- *"commits"?* It's `activityCount`, mislabelled.
- *Fix?* Rollup collection; move `sort + limit` into the pipeline; window by `timestamp`.

routes/analytics.js — computeStreak

- *What?* Consecutive days (user-local) with `$\Sigma \text{duration} > 0$` , anchored to today or yesterday, within a 90-day window.
- *Why "or yesterday"?* So a day that hasn't started yet doesn't read as a broken streak.
- *Complexity?* $O(N_{90})$ for the `$group`, $O(90)$ for the walk.
- *Edge cases tested:* stale-only activity → 0; 10 consecutive days → 10 (old code capped at 7); gap breaks it; UTC vs IST bucketing.
- *Bug risk?* Relies on the `?timezone=` offset being correct; DST transitions inside the window are not special-cased.

services/metricsService.js — buildMetrics

- *Why split service vs derive?* `metricsDerive` is pure and DB-free so it unit-tests without Mongo; `metricsService` does only the querying.
- *`$ifNull` everywhere?* Metrics depending on 2.1.0 fields must read missing as 0 for old activity.
- *`blocks: { $push: flowBlocksMs }` then flatten in JS?* `$push` of arrays → array of arrays; flattened with `reduce(concat)`. Could use `$reduce / $concatArrays` in the pipeline.
- *Not cached?* Correct — a known gap.

extension/src/editorTracker.ts — consumeChurn

- *What?* When lines are deleted, attribute them against still-fresh insertions (≤ 10 min) — newest first — and count the overlap as churn.
- *Why newest first?* "I just wrote this and immediately deleted it" is the churn signal; older insertions are more likely "real" code.
- *Complexity?* $O(R)$ over `recentInserts`, which is pruned to the 10-min window each call.
- *Edge case:* deletes with no matching insert $\rightarrow$ not churn (just `linesDeleted`).

extension/src/extension.ts — flushIfNeeded

- *Why stamp `timestamp` at interval start?* Stamping "now" pushed every session forward and skewed hour-of-day analytics (H-12).
- *What if `sendActivity` throws?* `bufferedMinutes = totalBuffered` (kept), `startedMs = now` — retried next tick. Memory only.
- *Race?* Single-threaded JS + one timer; `flushNow` and the tick can't truly overlap, but a slow `await` inside one flush while the timer fires again is possible — the second call sees `startedMs` already reset, so it's mostly benign.

frontend/src/pages/Dashboard.tsx — the three fetches

- *Why does it fetch 3 $\times$ on mount?* `useEffect([])` calls `fetchAnalytics` + `fetchWeekly`; `useEffect([viewModel])` also calls `fetchAnalytics` on the initial render $\rightarrow$ one redundant call (M-11). Fix: drop the mount effect, or guard on a "first render" ref, or use React Query.
- *Why `cache: 'no-cache'` + `Pragma`?* To defeat any intermediary caching of analytics — but it also defeats *useful* caching; React Query with a short `staleTime` is the right tool.

routes/goals.js — `/:goalId/progress`

- *Why `findOne({_id, userId})` not `findById`?* `findById` leaked another user's goal (title, target, deadline) — H-11.
- *Progress formula?* $\sum \text{activity.duration}$ (where `language === goal.techStack` && `timestamp  $\leq$  deadline`) / $3600 / \text{targetHours} * 100$, capped at 100.
- *Weakness?* Exact case-sensitive `language` match; `techStack` is free text ("React" vs "javascript"); no fuzzy/alias mapping.

25. Codebase map

Path	Purpose	Key functions/classes	Interview relevance
backend/app.js	Express bootstrap	route mounts, mongoose.connect , initScheduler , app.listen	"walk me through a request"; serverless mismatch
backend/api/index.js	Vercel adapter	serverless(app)	deployment
backend/config/passport.js	Google OAuth	GoogleStrategy verify, generateApiKey on new user	auth flow; where the key is born
backend/middleware/auth.js	Auth	isAuthenticated , verifyApiKey , bypassEnabled , resolveTestingUser	core — JWT vs API key, AUTH_BYPASS
backend/models/Activity.js	Activity schema + indexes	3 analytics sub-docs, 4 indexes	DB design; the missing timestamp index
backend/models/user.js	User schema	generateApiKey (randomBytes(32).hex)	the unique-key discussion
backend/models/Group.js / GroupMember.js	Groups	script password note; unique compound index	join-table modelling, concurrency
backend/models/Goal.js / Team.js / Notification.js	Other entities	embedded members[] on Team	embedding vs referencing
backend/routes/extension.js	Ingest	/track , /track/batch , /verify , normalizeTerminalAnalytics	payload contract, validation gaps
backend/routes/analytics.js	Dashboard reads	computeStreak , localDayInfo , buildTerminalSummary , resolveOwnedUserId	JS-vs-Mongo aggregation, timezones, streak
backend/routes/leaderboard.js	Global ranking	the all-collection aggregate + relative scoring	scalability (H-7), the rollup answer
backend/routes/metrics.js + services/metricsService.js + services/metricsDerive.js	Insights	buildMetrics , deepWorkRatio , consistencyIndex , truePeakWindow , estimationCalibration	"is this ML?"; pure-function testing
backend/routes/groups.js	Group lifecycle	create/join/leave/details, verifyPassword , opportunistic rehash	authz, concurrency, password handling
backend/routes/goals.js	Goals	/:goalId/progress owner scope	IDOR fix, progress formula
backend/routes/auth.js	OAuth + JWT	jwt.sign , cookie options	sessions vs JWT, CSRF
backend/routes/user.js	Profile + key	regenerate-api-key , complete-onboarding	key rotation
backend/routes/notifications.js	Notifications	all scoped by req.user._id	"the one that's done right"
backend/services/authorization.js	Authz helpers	sameUser , assertOwnership , isBypassAllowed	unit-tested, dependency-free
backend/services/passwordHash.js	Group pw hashing	crypto.scrypt , legacy plaintext	scrypt-over-bcrypt decision
backend/services/activityNormalizers.js	Ingest normalisers	normalizeEditor/Focus/GitAnalytics , MAX_FLOW_BLOCKS	additive schema evolution

Path	Purpose	Key functions/classes	Interview relevance
backend/services/notificationScheduler.js	Cron	checkUpcomingDeadlines , checkOverdueGoals , initScheduler	H-13 serverless problem
backend/tests/*.js	Tests	check() harness; static route scan; helper extraction	testing story
backend/server.js.old	Legacy monolith	had helmet + rate-limit ; per-day upsert model	"what changed and why"
extension/src/extension.ts	Extension entry	activate , flushIfNeeded , buildPayload , sendActivity , start	the whole §4
extension/src/editorTracker.ts	Editor metrics	EditorTracker , consumeChurn , sampleAttention	churn algorithm, complexity
extension/src/focusTracker.ts	Focus/flow	FocusTracker , tick , accrue	flow-block state machine
extension/src/gitStateTracker.ts	Git	GitStateTracker , handleStateChange	using another extension's API
extension/src/terminalTracker.ts + analyticsAggregator.ts + commandClassifier.ts + gitTracker.ts	Terminal	classifyCommand , sanitizeCommand , recordExecution	privacy (sanitisation), classification
extension/src/debugTracker.ts	Debug	DebugTracker	folding a dead pipeline into the live one
extension/tests/*.js	Extension tests	stub-driven; activation.test.js regression guards	the packaging-bug story
frontend/src/App.tsx	Shell + auth gate	loadUserProfile , NavLink	routing, auth state
frontend/src/pages/Dashboard.tsx	Analytics UI	fetchAnalytics/Weekly/TimeSlot , chart configs	charts, the mock-data panel, M-11
frontend/src/pages/Insights.tsx	Insights UI	fetchMetrics , hasFocusData gating	graceful degradation for old data
frontend/src/pages/Leaderboard.tsx	Leaderboard UI	team-average derivation	derived state in render
frontend/src/pages/Groups.tsx	Groups UI	tabs, modals, join/leave	client-side flow
frontend/src/pages/Goals.tsx	Goals calendar	client-only todos, createGoal	half-built feature
frontend/src/pages/Profile.tsx	Key management	regenerateApiKey , copy	rotation UX
frontend/src/contexts/ThemeContext.tsx	Theming	28 palettes, CSS vars, useTheme	context, localStorage, the verbose-styles trade-off
frontend/src/components/NotificationPanel.tsx	Notifications	30 s poll, browser Notification API, click-outside	polling vs push, the stale-closure bug (L-1)
docs/IMPROVEMENT_PLAN.md	Self-audit	13 H / 13 M / 7 L findings	shows engineering maturity — reference it in interviews
docs/TRACKING_ROADMAP.md	Metrics design	signal tiers, derived-metric definitions, the AI-layer plan	where Insights came from; the LLM design

26. "Defend your project" — 30 interviewer exchanges

1. Why MongoDB?

Activity records are append-only, self-contained, and the schema evolved three times without a migration — I added editor/focus/git analytics sub-documents and Mongoose just defaults missing ones. The access pattern is "one user's docs in a time window, then aggregate" — no joins on the hot path. Plus Atlas has a free tier and the team knew it.

2. But Mongo has weaker relational guarantees — is that a problem here?

Yes, in the group/team/goal parts. `groupmembers` I modelled as a proper join table with a unique compound index, which is the one place I do get a hard guarantee. But `team.members` is an embedded array with a lost-update window, and I have no cascades — `groups.js` has defensive "filter out nulls if the group was deleted" code that a foreign key would remove. For those parts Postgres would genuinely be better.

3. Why not Postgres then?

If I were starting again with the relational features in mind, I might. The activity stream itself is still better as documents or a time-series table. The honest answer is the relational parts are small and the document parts are the bulk and the hot path.

4. Your leaderboard — what happens at 100k users?

It breaks. It aggregates the entire `activities` collection and loads every user into Node on every request, with no window, cache, or pagination. The fix is a `userStats` rollup incremented on ingest so the leaderboard reads $O(\text{users})$ from an index, or a Redis sorted set for $O(\log n)$ rank lookups.

5. Why didn't you build the rollup already?

Time, and it wasn't the bottleneck at demo scale. It's the top item in my improvement plan. I'd rather ship the honest slow version and know exactly why it's slow than hand-wave.

6. Why not just cache the leaderboard?

A cache hides the cost, it doesn't remove it — the recompute still has to happen and it still scans everything. The rollup changes the complexity class; the cache is a nice addition on top for the top-N page.

7. Your API key — is that authentication?

It's a bearer credential. Possession authorises writes, there's no second factor, it's sent on every request. It's not "just an identifier" — an identifier would be safe to expose. It's an unscoped, non-expiring API token, and right now it's stored in plaintext.

8. What if someone steals another user's key?

They can forge unlimited activity for that user — inflate the leaderboard, corrupt the victim's analytics. They can't read the victim's dashboard, that needs the JWT cookie. Mitigation today is regeneration, which is all-or-nothing.

9. How would you fix the key model?

Hash it at rest with a `keyId` prefix so lookup is by an indexed id and comparison is constant-time on the hash. Support multiple named keys per device with independent revocation. Add rotation with a 24-hour grace window. Longer term, have the dashboard mint a short-lived signed ingest token the extension refreshes, or use an OAuth device flow so the extension never holds a long-lived secret.

10. Why not OAuth for the extension from the start?

A device flow is the right answer but it's more moving parts — a polling endpoint, token refresh, handling the "authorize in a browser" step inside VS Code. The API key was one `findOne` and worked offline. It's a legitimate MVP; I just wouldn't ship it to production unchanged.

11. Why not JWT between the extension and backend?

Same reason — a JWT needs an issue/refresh mechanism. If I keep the header model, I'd move to a short-lived JWT the dashboard issues, so at least it expires and is scoped.

12. Can users cheat the leaderboard?

Trivially. There's no rate limit or validation on ingest, so one `curl` with a valid key and `duration: 999999` puts you at #1. Defences: validate `duration` (1–3600s), rate-limit per key, require an idempotency key per flush, and run anomaly detection on the duration-vs-edits ratio.

13. What happens if the extension sends the same payload twice?

It's stored twice and double-counted — there's no idempotency. A client-generated key per flush plus a unique index would dedupe it.

14. What happens if two requests arrive simultaneously?

Ingest is a single-document insert, so Mongo makes it atomic — fine. The problematic concurrent cases are double-join (caught by the unique index, but I return a 500 instead of a 409) and `team.members.push` (a read-modify-write with a lost-update window — should be `$addToSet`).

15. Why aggregate analytics in Node instead of MongoDB?

Expedience while the metric definitions were still changing — `find().then(reduce)` is easy to debug. It's tech debt: it ships and parses every document and holds them all in memory. The metrics service already does it right with `$group` pipelines; the plan is to unify on that.

16. Why not calculate insights on the frontend?

The frontend would have to download every activity document to do it — huge payloads, and the raw data includes things I don't want to expose per-row. Aggregation belongs server-side near the data. The frontend just renders the numbers.

17. Is the Insights page machine learning?

No. It's deterministic statistics — coefficient of variation, a weighted score for the productive-hour ranking, medians, a ratio for estimation accuracy. I chose that so every number is explainable. An LLM narration layer is designed but not built, with a hard rule that any number it states must come from the metrics object.

18. Why use statistics instead of ML?

No labelled outcomes and not enough users to learn from. Descriptive stats work from the first data point, have no training pipeline, and are defensible in a code review. ML makes sense once I have goal-slippage or retention labels and a user base.

19. Why not use a simple rules engine for insights instead?

That's basically what `metricsDerive` is — thresholded rules on computed metrics (`consistencyIndex >= 0.6` → "steady"). The metrics are the rules layer.

20. Why not use an LLM for the insights?

I want to — for the narration. Not for the numbers, because it would hallucinate them. The design has a validator that rejects any figure the model outputs that isn't in the structured metrics. That's the piece I haven't built.

21. Why MongoDB aggregation over `$lookup` for the leaderboard join?

I can't `$lookup` cleanly because `activities.userId` is a `String` and `users._id` is an `ObjectId` — I coerce with `$toObjectId` in an `$addFields`. Migrating `userId` to `ObjectId` is on the list; then the join is native.

22. Why store everything in one `activities` collection instead of per-day buckets?

One insert per flush, immutable, no read-modify-write races, and I keep 10-minute granularity for the drill-down view. The cost is document count. The right long-term shape is: keep the raw stream and maintain bucketed rollups alongside it.

23. Why not Redis?

Nothing needs it yet — leaderboard and insights recompute in under a second at current scale. It's the obvious next dependency: a sorted set for the leaderboard, a cache for insights, and rate-limit counters.

24. Why not a message queue for ingest?

One write per user per minute is well within a single Mongo node. A queue earns its keep when ingest spikes threaten write latency or when I want to replay/repair — past ~10k concurrent users.

25. Why not WebSockets for notifications?

The `NotificationPanel` polls every 30 seconds, which is fine for deadline reminders. I'd add `WebSockets` for real-time group activity, not for this.

26. Why not microservices?

One developer, one deploy, one database, and the features genuinely share the data model. Splitting them would add network hops and distributed-transaction problems for no benefit. The one service I'd extract first is ingest, behind a queue, when volume demands it.

27. Your frontend doesn't build — `tsc -b` fails. Explain.

29 pre-existing TypeScript errors — mostly unused imports and variables, two real `string` not assignable to `never` cases. They predate my security work. `vite build` alone succeeds; the combined `tsc -b && vite build` script fails. It needs a cleanup pass; it's filed as M-13.

28. You have `@tanstack/react-query` installed but unused. Why?

It was added intending to adopt it and I didn't finish. Right now every navigation refetches with `cache: 'no-cache'`. Wiring React Query is a medium-term item — it'd give caching, dedup, and retry with almost no new code.

29. How do you know your aggregation pipelines are correct if nothing ran against a real DB?

I don't, fully — that's stated in the session log. The pure functions (`metricsDerive` , normalisers, authorization, streak helpers) are unit-tested, and `streak.test.js` extracts the shipped code rather than a copy so it catches regressions. But there's no integration test booting Express against a real Mongo. That's the biggest testing gap and it's on the plan — `supertest` + `mongodb-memory-server` .

30. If this got 10,000 real users tomorrow, what's your first move?

Add the `{userId:1,timestamp:-1}` index and put a 30-day window + `$limit` on the leaderboard pipeline as a stopgap, then build the `UserStats` rollup. In parallel: `helmet` + rate limiting so ingest can't be flooded, and `express-validator` so duration can't be faked. Then a `/health` that fails readiness when Mongo is down, and a real logger so I can see what's happening.

27. Final self-audit

- ✓ Whole repository inspected — backend (all routes/models/services/tests), extension (all `src/*.ts` + tests), frontend (all pages + key components + context), config, docs.
- ✓ Extension traced end to end: activation → trackers → flush loop → payload → POST.
- ✓ Extension → API → MongoDB path traced (`verifyApiKey` → normalisers → `Activity.create`).
- ✓ Every model examined; index gap identified (`timestamp` vs `date`).
- ✓ Every meaningful API endpoint tabulated with auth + processing.
- ✓ Frontend architecture, state model, and the three real bugs (mock panel, unpersisted todos, orphaned Teams) documented.
- ✓ Leaderboard analysed (O(A+U), relative scoring, email exposure) + the rollup answer.
- ✓ Groups analysed (join table, membership check, double-join → 500, no admin role).
- ✓ Insights analysed in full — confirmed **statistics, not ML**; LLM layer designed not built.
- ✓ Auth analysed — Google OAuth → JWT cookie (web), API key (extension), no refresh, no revocation, `JWT_SECRET` fallback.
- ✓ Unique key analysed — classified as an unscoped non-expiring plaintext bearer credential; attack scenarios + redesign.
- ✓ Security: separated fixed (H-1/2/9/10/11) from open (keys, rate limit, headers, validation, error leakage, email exposure, idempotency, regex ReDoS).
- ✓ Scalability: 10k / 100k / 1M analysis with concrete evolution.
- ✓ Limitations: current vs production, each with impact + fix.
- ✓ No invented implementation details — everything marked ACTUAL or RECOMMENDED.
- ✓ Source files referenced throughout + a codebase map.
- ✓ Deliverables created: this file, `CodeTrackr_Interview_QA.md` , `CodeTrackr_Architecture.md` , `CodeTrackr_Interview_Cheat_Sheet.md` , `../../CODETRACKR_PROJECT_CONTEXT.md` , and the PDF.

Biggest interview risks to rehearse

1. **The API key** — know the identifier-vs-credential distinction cold, and the redesign.
2. **The leaderboard scan** — know the exact complexity and the rollup / sorted-set answer.
3. **"Is it ML?"** — say no, confidently, and explain why that was the right call.
4. **The broken frontend build** — own it, it's a cleanup not a design flaw.
5. **Nothing tested against a real DB** — own it, name the fix (`supertest` + memory server).
6. **"What did you build?"** — describe what you can defend in depth; don't over-claim.